Hoofdstuk 4

Het datawarehouse

In het vorige hoofdstuk is uiteengezet waarom een datawarehouse nodig is voor een succesvolle implementatie van Business Intelligence. In dat hoofdstuk zijn veel problemen genoemd die kunnen optreden zonder datawarehouse. In alle gevallen is gezegd dat een centraal datawarehouse de (of een deel van de) oplossing is.

In dit hoofdstuk gaan we kijken naar het ontwerp van een datawarehouse. We zullen uitgebreid ingaan op dimensioneel modelleren, een techniek die bij de meeste implementaties wordt toegepast. Naast het dimensioneel modelleren komt de methode van Inmon aan bod, alsmede de Data Vault.

4.1 Dimensioneel modelleren

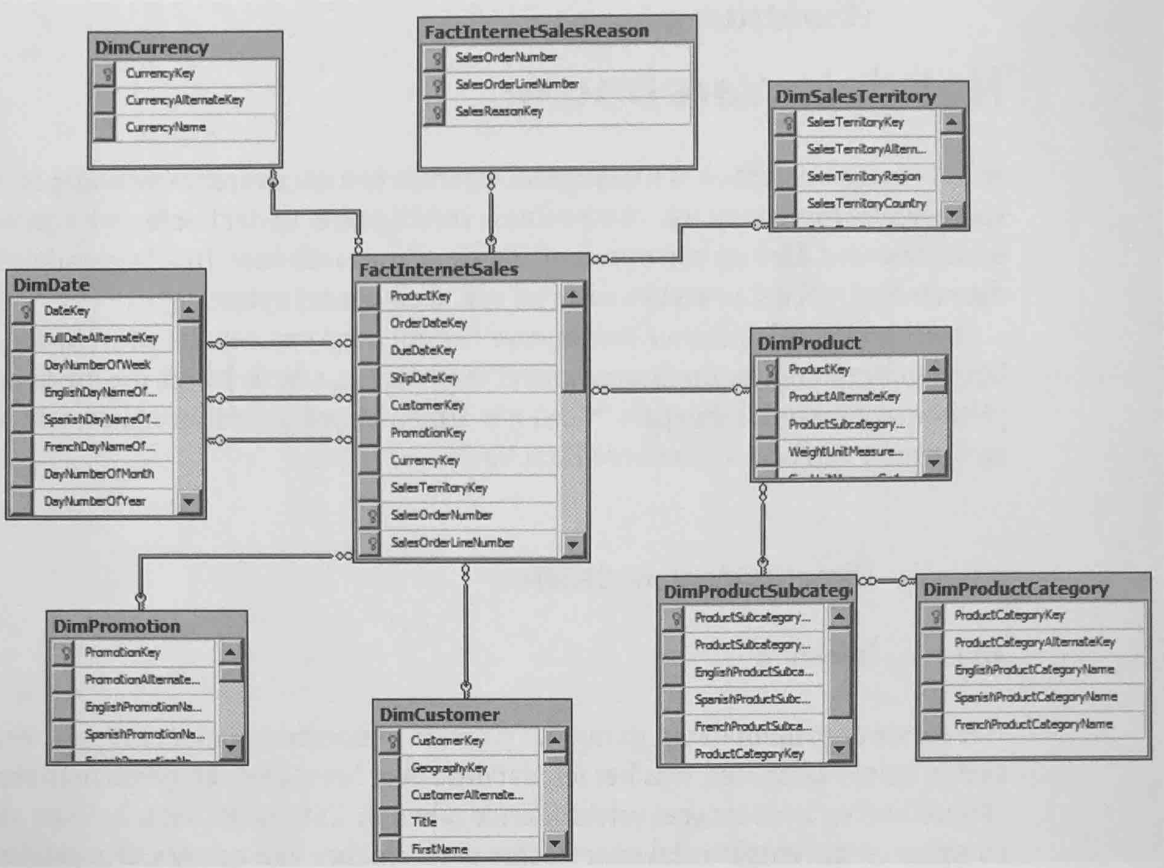
4.1.1 Inleiding

Het eerste argument dat in paragraaf 3.1 werd genoemd om niet te rapporteren op operationele databases, was het feit dat deze over het algemeen genormaliseerd zijn. Normaliseren leidt tot veel verschillende tabellen. Dit maakt het schrijven van query's complex en het zorgt veelal voor slechte performance van query's. Dimensioneel modelleren is een manier van databaseontwerp, die poogt deze tekortkomingen weg te poetsen. Uitgangspunt is dat de database een voornamelijk alleen-lezen-workload (read-only) krijgt. Voor een database met een OLTP-workload is dimensioneel modelleren niet geschikt.

Dimensioneel modelleren leidt tot een database met een zogenaamd stermodel. In figuur 4.1 zie je een voorbeeld van een stermodel. Een database die bestaat uit slechts één stermodel bevat informatie over één proces binnen een organisatie. Een database die het bij één ster laat, wordt een **datamart** genoemd. In de theorie van Kimball is het datawarehouse de verzameling van alle stermodellen, die op hun beurt alle businessprocessen beschrijven.

Een **datamart** is een database waarvan de tabelstructuur een ster vormt en die één proces uit de organisatie beschrijft.

In de theorie van Kimball bestaat het datawarehouse uit de verzameling van alle stermodellen die tezamen de hele organisatie beschrijven.



Figuur 4.1 *Voorbeeld van een stermodel*

Een stermodel bestaat uit twee soorten tabellen:
- de feitentabel;
- dimensietabellen.

Het stermodel ontleent zijn naam aan het feit dat de feitentabel centraal staat en de dimensietabellen er in een soort sterstructuur omheen staan.

De feitentabel

Centraal in figuur 4.1 staat de tabel factInternetSales. Dit is de feitentabel. In het voorbeeld zie je een stermodel dat een verkoopproces weerspiegelt. Als je aan een verkoopmanager vraagt hoe het gaat, zal hij dingen zeggen als: Het gaat goed want de *omzet* is x miljoen. Het gaat niet naar verwachting want de *marge* is te laag. Het *aantal verkochte* producten is weliswaar gestegen, maar de *kosten* die we daarvoor hebben moeten maken zijn harder gestegen. Omzet, marge, aantal stuks verkocht en kosten zijn allemaal meetbare grootheden die iets zeggen over het verkoopproces. Het zijn de getallen die het proces meetbaar hebben gemaakt. Of anders gezegd, het zijn de feiten. Deze feiten sla je op in de feitentabel, de tabel die centraal staat in het stermodel.

De **feitentabel** is de tabel waar grootheden die een proces meetbaar maken in worden opgeslagen. KPI's worden gebaseerd op deze grootheden.

Dimensietabellen
Feiten alleen zijn niet genoeg, getallen zeggen niets. Getallen zullen altijd in context geplaatst moeten worden. Dat de omzet één miljoen euro bedraagt, zegt eigenlijk nog niets. De volgende zin heeft wel betekenis: één miljoen omzet aan Business Intelligence-cursussen in het afgelopen half jaar. De betekenis komt van de context. Opeens gaat het over een concreet product in een concrete periode. De context, de betekenis, komt van de dimensies. In het voorbeeld zie je onder andere een product-dimensie (dimProduct) en een datum dimensie (dimDate) die de context van het voorbeeld bevatten.

Dimensietabellen zijn de tabellen die de context bevatten die betekenis geeft aan de feiten.

Het idee achter het stermodel is dat iemand die het proces kent, het stermodel snapt. Iedereen op de afdeling verkoop weet dat er producten verkocht worden en welke dat zijn. Iedereen weet dat er targets (doelstellingen) zijn geformuleerd per periode. Met die kennis wordt het plaatje eenvoudig te lezen. Daarmee wordt de database eenvoudig te gebruiken. Met als bijkomend voordeel: als het voor mensen eenvoudiger wordt, geldt dat over het algemeen ook voor de computer, waardoor de performance verbetert.

Een **stermodel** is voor een vakinhoudelijk deskundige een gemakkelijk te begrijpen, en daarmee te gebruiken, model.

4.1.2 Modelleren

Ralph Kimball is de grondlegger van het dimensioneel modelleren. Een van de standaardwerken over dimensioneel modelleren is het boek van Ralph Kimball en Margy Ross *The Data Warehouse Toolkit*. In dit boek beschrijven zij vier hoofdstappen om tot een stermodel te komen. Deze stappen zijn:
1. Kies het te modelleren proces.
2. Bepaal het te gebruiken detailniveau.
3. Bepaal de van toepassing zijnde dimensies.
4. Bepaal de relevante feiten.

We zullen deze stappen een voor een kort bespreken.

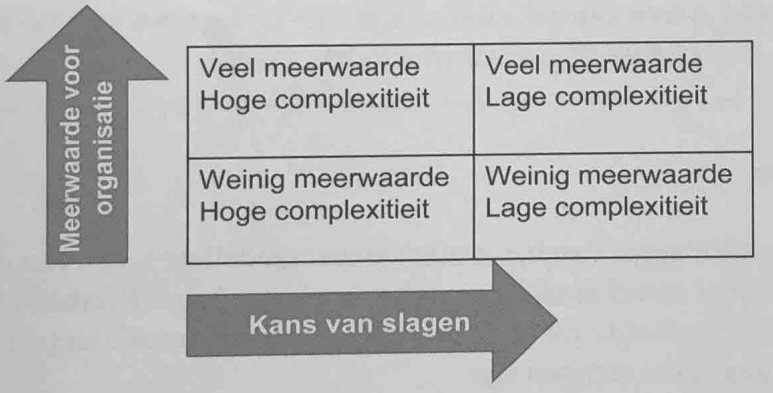
Kies het te modelleren proces
De eerste stap in het komen tot een dimensioneel ontwerp, is het kiezen van het te modelleren proces. Hieruit volgt meteen een heel belangrijk kenmerk van het dimensioneel modelleren: wat je gaat modelleren is een proces. Je vangt een proces in een databasemodel. Het proces staat dus centraal. Niet voor niets is er in de eerste hoofdstukken op gehamerd dat Business Intelligence geen ICT-'ding' is, noch een technisch iets. Het gaat om de bedrijfsvoering, om de processen.

Vanuit de tot nu toe behandelde stof klinkt het bovenstaande hopelijk logisch. Vanuit een klassieke database-invalshoek is het echter niet vanzelfsprekend. Veel

mensen leren tijdens hun opleiding dat je een database moet normaliseren, een term die in hoofdstuk 3 ook is gebruikt. Bij normaliseren kijk je voornamelijk naar de structuur van de gegevens. Je modelleert de gegevens en hun onderlinge verbanden, niet zo zeer het proces. Een dimensioneel gemodelleerde database staat daarmee dichter bij de gebruikers, waar door de database makkelijker te gebruiken wordt. Daarmee is al een belangrijke doelstelling behaald.

Bij **normaliseren** modelleer je de gegevens en hun verbanden. Bij *dimensioneel modelleren* modelleer je de bedrijfsprocessen. Dimensioneel modelleren staat dichter bij eindgebruikers.

Het kiezen van het proces dat je gaat modelleren kan gemakkelijk zijn of juist niet. Soms is de vraag waaruit het project voortkomt expliciet verbonden aan een proces. Bijvoorbeeld: het verkoopproces moet geoptimaliseerd worden en daartoe hebben we meer en betere inzichten nodig in dat verkoopproces. Het kan ook zijn dat je uit de informatieanalyse heel veel verschillende soorten informatiebehoeftes hebt gedestilleerd die allemaal te maken hebben met andere zaken binnen de organisatie. Het is dan van belang te beseffen dat je voor elk apart proces een apart stermodel moet maken. De ster modelleert immers het proces. Maar waar moet je beginnen?



Figuur 4.2 *Beslismatrix*

Figuur 4.2 is een herhaling van de afbeelding die je gezien hebt bij het verhaal over scope in hoofdstuk 2.

Sommige processen zijn heel complex. Complexe processen met veel uitzonderingen zijn moeilijk te modelleren. Daarmee neemt het risico op fouten in het model toe. Fouten in het model leiden tot fouten in de rapportages. Fouten in de rapportages leiden ertoe dat de gebruikers de datamart niet vertrouwen en dus niet gaan gebruiken. Project mislukt.

Er zijn ook relatief makkelijke processen. Gemakkelijk is subjectief en kan te maken hebben met de kennis en ervaring van de betrokken gebruiker en/of met de ervaring van degene die het model moet maken. Duidelijk is dat de kans op succes toeneemt terwijl als extra bonus de benodigde tijd om de database te bouwen afneemt.

Start het bouwen van het datawarehouse met het modelleren van een proces dat zich in de rechterbovenhoek van het kwadrant van figuur 4.2 bevindt: gemakkelijk en met veel meerwaarde!

Bepaal het te gebruiken detailniveau

Nadat het te modelleren proces is gekozen, moet je nadenken over het gewenste detailniveau, ofwel de grain. Deze 'keuze' volgt uiteraard rechtstreeks uit de informatieanalyse. Toch is het wenselijk om hier goed over na te denken, voornamelijk vanuit een performance-overweging.

De **grain** is het detailniveau van de datamart.

Het detailniveau dat je kiest bepaalt hoe groot de database gaat worden. Het heeft daarmee ook rechtstreeks invloed op de performance die je mag verwachten. Aan de andere kant van de medaille bepaalt het welke vragen wel en niet beantwoord kunnen worden. Je kan nooit gedetailleerdere vragen beantwoorden dan die van de grain van de datamart.

Voorbeeld

Stel dat een internetwinkel gemiddeld honderd orders per dag krijgt en dat er per order gemiddeld twee producten worden gekocht. Met een maand van dertig dagen levert dat `100 x 2 x 30 = 6.000` records op per maand. Met tien jaar aan historie in de database, zijn dat 720.000 records.
Als je als grain maandomzet kiest, dus één record per maand, houd je nog maar 120 records over.

Bovenstaand voorbeeld is wat overtrokken om het punt duidelijk te maken. Uiteraard verlies je heel veel informatie met maand als laagste detailniveau. In een maand worden veel verschillende producten verkocht. Je kunt niet meer analyseren welke producten hoeveel omzet hebben opgebracht. Hetzelfde geldt voor klanten. Hoeveel klanten hebben orders besteld en hoeveel besteedt een klant gemiddeld per maand? Dat zijn allemaal vragen die niet meer beantwoord kunnen worden. Financiële rapportages met omzet per maand, kwartaal of zelfs jaar zijn razend snel.

Een laag detailniveau resulteert in een grote database met bijbehorende performance-uitdagingen, maar levert wel de meeste mogelijkheden in termen van welke vragen beantwoord kunnen worden.

Met de huidige hardware en met de huidige verwachtingspatronen van gebruikers is het laatste argument vele malen zwaarder dan het eerste. Mensen gaan op zoek naar meer details. Als dat niet mogelijk blijkt, raken ze teleurgesteld in het datawarehouse.

Dat is slecht voor de acceptatie ervan. Bovendien zijn de kosten van het opslaan van miljoenen records tegenwoordig niet meer zo groot. Houd er wel rekening mee dat performance een probleem kan worden als de tabellen heel erg groot worden. Ook een slechte performance kan leiden tot slechte acceptatie van het datawarehouse. In hoofdstuk 5 zul je nog enkele trucs leren om de performance goed te krijgen.

Het te kiezen detailniveau volgt als het goed is uit de informatieanalyse, uit de behoeften die bestaan binnen de organisatie. Bij twijfel moet je terug naar de gebruikers. Veel (alles) opslaan heeft over het algemeen wel de voorkeur vanwege de sterk afgenomen kosten die dat met zich meebrengt en de daardoor toenemende mogelijkheden van het datawarehouse.

Alle beschikbare details opslaan heeft tegenwoordig de voorkeur.

Volgens Kimball is het resultaat van deze stap het zogeheten grain statement. Een grain statement zou kunnen zijn: 'Verkopen moeten worden geanalyseerd per product, per klant en per dag'.

Het **grain statement** is de uitspraak die vastlegt op welk detailniveau je gegevens in het datawarehouse gaat opslaan.

Bepaal de van toepassing zijnde dimensies

In de inleiding hebben we het begrip dimensie geïntroduceerd. Dimensies geven context, betekenis, aan de cijfers die een proces inzichtelijk maken. Een getal alleen, zonder context, betekent niets. Het is de context die de betekenis bepaalt.

De belangrijkste dimensies volgen vanzelf uit het grain statement. Bovenstaand grain statement brengt ons bij de dimensies Product, Klant en Datum. Waarschijnlijk zijn er meer dimensies dan die welke rechtstreeks uit het grain statement volgen.

De belangrijkste dimensies volgen uit het grain statement.

Het kan voorkomen dat je bovenstaand grain statement hebt geformuleerd. Uit de informatieanalyse komt ook naar voren dat mensen zitten met vragen zoals: Gaat de omzet omhoog als we een iPad weggeven bij elke aankoop van boven de € 500,-? En zo ja, geldt dat ook voor de winst, gezien de toegenomen kosten?

Dergelijke vragen spelen zich niet op een lager detailniveau af dan de gedefinieerde grain. Ze leiden wel tot de introductie van een nieuwe dimensie. In figuur 4.1 is het de tabel dimPromotion die deze dimensie vertegenwoordigt.

Soms leidt de introductie van een dimensie wel tot de noodzaak een andere grain te kiezen. Deze en de voorgaande stap zijn dan ook iteratief. Je blijft ze net zolang herhalen totdat je alle dimensies hebt gevonden.

Het bepalen van het grain statement en het bepalen van de relevante dimensies is een iteratief proces.

Dat brengt ons op de vraag wanneer iets een dimensie is.

Stel dat je uit de informatieanalyse de volgende opmerking hebt gehaald: 'Een inkoper vermoedt een verband tussen de maat van producten en de kleur'. Kopen kinderen vaker producten in felle kleuren terwijl volwassenen meer voor de wat neutralere kleuren gaan? Blijft dit stabiel of heeft mode en tijdgeest hier invloed op? Om beter te kunnen inkopen wil de inkoper deze vragen graag beantwoord zien.

Heb je zojuist twee nieuwe dimensies gevonden, kleur en maat? In dit geval zijn kleur en maat kenmerken van de dimensie Product. Een dimensie kenmerkt zich door de vele beschrijvende elementen die ervoor te vinden zijn. Van een product weet je zaken als inkoopprijs, verkoopprijs, levertijd, gewicht, maat en kleur. Van kleur en maat zelf weet je niets. Ze beschrijven een product maar hebben zelf geen beschrijvende kenmerken. Daarmee worden het kolommen in de tabel dimProduct in plaats van dat het zelf dimensies (en dus tabellen) worden. We noemen kleur en maat ook wel attributen.

> Een dimensie kenmerkt zich door de vele beschrijvende elementen die ervoor te vinden zijn.

Verderop in dit hoofdstuk leer je meer over dimensies en het ontwerpen van dimensies.

Bepaal de relevante feiten

De vierde en laatste stap in het door Kimball beschreven vierstappenplan is het bepalen van de relevante feiten. Feiten worden ook wel meetwaarden (measures) genoemd. De processen, en veranderingen in die processen, moeten meetbaar zijn. Je wilt doelstellingen definiëren en kunnen meten of die doelstellingen gehaald zijn. Doelstellingen moeten SMART zijn, waarbij de M staat voor meetbaar.

Een verkoopproces is meetbaar door het aantal verkochte producten te tellen, de omzet te berekenen, de kosten en de marge te bekijken et cetera. Dat zijn de feiten, ofwel de meetwaarden. Deze feiten worden kolommen in de feitentabel, de kern van het stermodel.

In paragraaf 4.1.7 leer je meer over feiten en feitentabellen.

Beperkingen

Je hebt zojuist de vier stappen geleerd die je moet doorlopen als je een stermodel wilt ontwerpen. In theorie worden de keuzes die je tijdens dit stappenplan maakt volledig bepaald door de informatieanalyse. De gebruikers hebben kenbaar gemaakt wat ze aan informatie nodig hebben. Dat bepaalt het nodige detailniveau, de dimensies en dimensiekenmerken, en de feiten.

Een belangrijke stap die tot nu toe onderbelicht is gebleven, is de terugkoppeling naar de gebruikte operationele systemen. Als bij een verkoop niet wordt vastgelegd of er ten tijde van die verkoop een actie gaande was, is het verhaal over van toepassing zijnde dimensies slechts theorie. Alles wat de primaire systemen niet registreren, is (waarschijnlijk) niet voorhanden.

Een dergelijke discrepantie tussen wat de gebruikers graag zouden willen en wat op basis van de bestaande gegevens mogelijk is, kan ertoe leiden dat de bestaande operationele databases als uitgangspunt gaan dienen voor het ontwerpen van je dimensionele model. Zoals uitgebreid besproken in hoofdstuk 2, is het een valkuil om Business Intelligence te zien als een technisch iets dat van ICT is. Met de bestaande databases als uitgangspunt dreigt die valkuil. Laat altijd de wensen en eisen van de gebruikers leidend zijn in je opzet. Tegelijkertijd, beloof nooit iets dat je niet kunt waarmaken.

Laat altijd de wensen en eisen van de gebruikers leidend zijn in je opzet.

4.1.3 Dimensies

Zoals gezegd geven de dimensies betekenis aan de feiten. Daarmee zijn de dimensies misschien wel de belangrijkste component van een dimensioneel model. Ze bepalen de doorsnedes die de gebruiker kan maken. Een veel gebruikte term in datawarehousing is *'to slice and dice'*. Waarbij *to slice* hier staat voor de mogelijkheid te filteren. Beide komen neer op de wens de cijfers vanuit verschillende invalshoeken te kunnen bekijken.

To slice and dice staat voor het maken van willekeurige doorsnedes door de feiten.

Bij het bepalen van wat dimensies zijn, kijk je in het verslag van de informatieanalyse naar zinnen als:

- In het verkoopproces moet de omzet uitgesplitst kunnen worden naar productcategorieën.
- De gegevens moeten getoond kunnen worden voor een specifiek jaar.
- Per verkoper moeten de orders en de resulterende omzet bekeken kunnen worden.

Hoewel telkens verschillend geformuleerd, komt het iedere keer op hetzelfde neer: omzet per categorie, per jaar, per verkoper. Overal waar het woordje 'per' om de hoek komt, hebt je te maken met een dimensieattribuut of met een dimensie.

Een **attribuut** is een kenmerk van een dimensie, ofwel een kolom in de dimensietabel.

Uit de eerste twee genoemde zinnen volgen productcategorie en jaartal als dimensieattributen van de dimensie Product en de dimensie Datum. Je weet niet zozeer iets over categorie of jaartal. Categorie zegt iets over een product, jaartal zegt iets over een periode.

De derde zin is iets algemener geformuleerd. Verkoper is hier een dimensie en niet een attribuut. Van een verkoper zijn veel kenmerken denkbaar, zoals naam, geboortedatum en datum in dienst. Deze kenmerken zijn de attributen van de dimensie Verkoper.

In technische termen komt het op het volgende neer. Alles waarop gefilterd of geaggregeerd moet worden, moet een attribuut zijn van een dimensie. Een attribuut is een kolom die gebruikt kan worden in de `WHERE`-clausule of in de `GROUP BY`-clausule van een query.

- Nadat je de relevante dimensies hebt onderkend, is het zaak zo veel mogelijk relevante beschrijvende elementen (attributen) van deze dimensies te benoemen.

Bijna elk datawarehouse heeft een datumdimensie. In bijna alle analyses die mensen doen, speelt de tijd een rol. Daarom zullen we hier nu uitgebreid aandacht besteden aan de datumdimensie.

Waarom een datumdimensie?
Bijna alle datewarehouses hebben een datumdimensie. In eerste instantie is dat ook erg voordehandliggend. In zo goed als alle gevallen speelt de tijd een rol in onze analyses. Voor een buitenstaander zegt het weinig tot niets als een bedrijf afgelopen jaar een omzet heeft gedraaid van 15 miljoen euro. De directeur heeft hier waarschijnlijk een gevoel bij: 15 miljoen is goed, of juist niet. Dat gevoel komt voort uit voorkennis, hij weet wat de omzet van het jaar ervoor was en wat de verwachting voor het afgelopen jaar was. Het gevoel komt dus uit historische kennis. De tijd speelt een belangrijke rol.

- Elk datawarehouse heeft een datum/tijddimensie omdat de tijd een belangrijke rol speelt in ons leven.

Een belangrijk doel van Business Intelligence is om gevoel dat voortkomt uit ervaring te staven met harde feiten. Daaruit vloeit voort dat het belangrijk is zaken als omzet te kunnen afzetten tegen periodes. Zo kunnen periodes vergeleken worden en groei en krimp berekend worden. Er is bijna geen bedrijf dat zonder dit soort vergelijkingen kan. Daarom heeft elke datamart een datumdimensie. In figuur 4.3 zie je een voorbeeld van een datumdimensie.

DateKey	FullDateAlterna...	DayNumberOf...	EnglishDayNam...	DayNumberOf...	DayNumberOfY...	WeekNumberO...	EnglishMonthN...	MonthNumberO...	CalendarQuarter	Calend
20050101	2005-01-01	7	Saturday	1	1	1	January	1	1	2005
20050102	2005-01-02	1	Sunday	2	2	2	January	1	1	2005
20050103	2005-01-03	2	Monday	3	3	2	January	1	1	2005
20050104	2005-01-04	3	Tuesday	4	4	2	January	1	1	2005
20050105	2005-01-05	4	Wednesday	5	5	2	January	1	1	2005
20050106	2005-01-06	5	Thursday	6	6	2	January	1	1	2005
20050107	2005-01-07	6	Friday	7	7	2	January	1	1	2005
20050108	2005-01-08	7	Saturday	8	8	2	January	1	1	2005
20050109	2005-01-09	1	Sunday	9	9	3	January	1	1	2005

Figuur 4.3 *Voorbeeld van een datumdimensie*

Maar is het wel zo logisch een datumdimensie te bouwen? Als je in de feitentabel een kolom opneemt met daarin de datum van wanneer dat feit optrad, dus de orderdatum als het om een order-feitentabel gaat, dan ben je toch klaar? Met die datum kun je bere-

kenen wat je nodig hebt: het jaartal, het kwartaal, de maand, de naam van de dag en ga zo maar door. Waarom heb je dan nog een datumdimensie nodig?

Er is een aantal redenen waarom datumdimensies veel meerwaarde hebben in datamarts. Niet alle redenen zijn voor elke situatie van toepassing, dat moet je per project bekijken. De voordelen van een aparte datumdimensie zijn:

- gemak van filteren;
- gebroken boekjaren;
- weeknummers;
- ontbrekende data;
- extra metadata;
- performance.

Gemak van filteren – In eerdere hoofdstukken is de vraag wie rapporten maakt, of beter gezegd, wie de gebruiker van het datawarehouse is, al ter sprake gekomen. Steeds vaker is de gebruiker iemand die een functie binnen het proces heeft en dus niet een ICT'er. Hoe goed is deze gebruiker in staat om uit een datum het kwartaal te halen? Jij weet misschien dat dat kan met de `DATEPART`-functie van SQL. Het onderstaande voorbeeld geeft het kwartaal terug waarin de OrderDate valt:

```
DATEPART( QUARTER, [Orderdate])
```

Niet alle gebruikers van het datawarehouse zullen over deze kennis beschikken.

Ook de gebruikte tool om query's te schrijven speelt een rol. Steeds meer worden drag-and-droptools gebruikt. Als een gebruiker de omzet per kwartaal wil zien, sleept hij simpelweg de kolommen Omzet en Kwartaal naar het rapport. Klaar! Tenminste, als er een kolom Kwartaal is.

Met een datumdimensie kun je ervoor zorgen dat er voor elke relevante periode een gedefinieerde kolom bestaat.

Gebroken boekjaren – Veel bedrijven hebben zogenaamde gebroken boekjaren. Dat wil zeggen dat een boekjaar niet loopt van 1 januari tot en met 31 december, maar bijvoorbeeld van 1 juli tot en met 30 juni. Dit heeft vaak tot gevolg dat sommige rapportages 'gewoon' de kalender volgen, maar dat andere rapportages gebaseerd moeten zijn op boekjaren. Voor een specifieke order kan dan gelden dat deze valt in kalenderjaar 2017, maar in boekjaar 2018.

Wanneer een boekjaar begint en eindigt is een bedrijfsregel (business rule). Je wilt geen bedrijfsregels opnemen in rapporten. De kans dat er fouten worden gemaakt is te groot en de controleerbaarheid en onderhoudbaarheid zijn te slecht.

Een datumdimensie met een kolom voor kalenderjaar en een aparte kolom voor fiscaal jaar of boekjaar biedt een goede structurele oplossing. Evenzo maak je kolommen voor kalenderkwartaal en fiscaal kwartaal en alle andere van toepassing zijnde periodes. De gebruiker van het datewarehouse hoeft nu alleen maar de juiste kolom te gebruiken in zijn analyse.

Dé datumdimensie bevat de bedrijfsregel die definieert van wanneer tot wanneer een (gebroken) boekjaar loopt.

Weeknummers – Een ander veel terugkerend probleem is het weeknummer. Is week 1 nou de week waarin 1 januari valt, de week waarin ten minste 4 dagen van januari voorkomen of de eerste volle week in januari? Er zijn veel definities mogelijk en veel mensen weten eigenlijk niet eens welke definitie de standaard is in Nederland. In de datumdimensie kun je voor elke dag opnemen in welke week die dag valt.

De datumdimensie bevat de bedrijfsregel die definieert in welk weeknummer een dag valt.

Ontbrekende data – Stel je maakt een datamart voor een kleine ICT-opleider. Omdat er in de zomer bijna niemand op cursus komt, is de opleider de hele maand augustus gesloten. Stel verder dat je geen datumdimensie hebt maar alleen een kolom met de startdatum van elke cursus. Als iemand met deze gegevens een staafgrafiek maakt om de omzet per maand te tonen, ontbreekt de hele maand augustus. De grafiek heeft maar elf balken terwijl een jaar twaalf maanden kent.

Ook hier geldt weer dat een datumdimensie de oplossing kan bieden. Voor elke periode die op de kalender voorkomt maak je een record, ongeacht of je in die periode zaken hebt gedaan. Feiten of niet, een dag is een dag. En het feit dat je in een bepaalde periode geen business hebt gehad, is informatie op zich.

Een datumdimensie bevat gegevens over alle periodes, ook periodes waarin zich geen feiten hebben voorgedaan.

Extra metadata – Los van hele dagen die in operationele databases niet voorkomen, is er vaak extra kennis over een dag nodig. Is een dag een werkdag? Zo niet, is het weekend, of is het een vakantiedag zoals Koningsdag? Je kunt een datumdimensie verrijken met dit soort additionele informatie. De gebruiker kan nu het effect van vakantiedagen op de omzet analyseren.

Een datumdimensie bevat extra informatie over periodes, bijvoorbeeld of een dag een werkdag is.

Performance – Tot slot is er nog een performancereden om een datumdimensie te maken. Deels heeft dat te maken met het eerste argument. In plaats van voor elke query opnieuw met functies zoals de getoonde `DATEPART` een periode te berekenen, is die berekening eenmalig, vooraf, gedaan. Dat spaart rekentijd. Bovendien is er een grote kans dat de database-engine, het proces dat de query moet uitvoeren, efficiënter kan werken met de kolommen uit de datumdimensie. Je query's worden dus sneller.

Hoe bouw je een datumdimensie?
Nu je weet waarom je een datumdimensie nodig hebt in een datamart, is het tijd om er één te maken. Dat doe je in drie stappen:
1. bepaal het gewenste detailniveau;
2. bepaal de sleutels;
3. bepaal de overige kolommen.

Bepaal het gewenste detailniveau – De eerste stap is het bepalen van het gewenste detailniveau. Als het goed is volgt dat uit het grain statement van het stermodel. In de praktijk zijn veel datumdimensies op dagniveau. Dat houdt in dat je tot op dagniveau analyses kunt doen. Je maakt hiertoe voor elke dag op de kalender een record in de datumdimensie.

In sommige datamarts moeten de feiten tot op de seconde terug geanalyseerd kunnen worden. Dit houdt volgens hetzelfde principe in dat je voor elke mogelijke seconde een record aanmaakt. Als je 10 jaar aan historie wilt gaan opslaan, krijg je ruim 315 miljoen records. Waarschijnlijk is het aan te raden een aparte datum- en tijddimensie aan te maken. Deze bevatten respectievelijk ruim 3600 en ruim 86.000 records.

In veel datawarehouses bevat de datumdimensie een record voor elke dag op de kalender.

Bepaal de sleutels – De volgende stap is het bepalen van de sleutels. In eerste instantie moet je een primaire sleutel (primary key) kiezen. Het is slim om daarnaast ook sleutels te maken voor periodes.

Voor de datumdimensie is het een best practice een zogenaamde smart key te gebruiken. Een smart key is niet een betekenisloos nummertje maar bevat echte informatie. De sleutel van een datumdimensie ziet er bij voorkeur uit zoals in onderstaand voorbeeld:

```
20170611
```

Dit is een geheel getal en wordt ook als zodanig opgeslagen in de datumdimensie. Het vergt niet veel fantasie om in dit getal de datum 11 juni 2017 te herkennen. Het voordeel van een sleutel als deze is dat gehele getallen efficiënter zijn in opslag en een betere performance opleveren dan datums. Door eerst het jaartal op te nemen en dan pas maand en dag, blijft wel de juiste sorteervolgorde gehandhaafd.

Stel nu dat je in je datumdimensie een kolom Kwartaal hebt opgenomen en dat daarin de getallen 1 tot en met 4 kunnen voorkomen. In figuur 4.4 kun je zien wat er gebeurt als iemand een draaitabel maakt met de kwartalen op de horizontale as.

	A	B	C	D	E	F
1	Internet Sales Amount	Column Labels				
2	Row Labels	1	2	3	4	Grand Total
3	⊞Accessories	$173,550.63	$199,754.98	$152,419.17	$175,035.18	$700,759.96
4	⊞Bikes	$7,229,254.22	$8,776,282.57	$5,420,034.74	$6,892,573.11	$28,318,144.65
5	⊞Clothing	$86,053.86	$98,374.79	$73,083.70	$82,260.26	$339,772.61
6	Grand Total	$7,488,858.71	$9,074,412.34	$5,645,537.61	$7,149,868.55	$29,358,677.22

Figuur 4.4 *Draaitabel*

Los van het feit dat een kolom met daarin het getal 1 niet zoveel zegt – Q1 of kwartaal 1 was beter geweest – zijn alle feiten waarvoor geldt dat de kolom Kwartaal een 1 bevat, bij elkaar opgeteld. Ongeacht het jaartal. De getoonde draaitabel vergelijkt de omzet per kwartaal over alle jaren heen. Misschien dat dit de bedoeling is, maar de kans is groot dat de gebruiker twaalf kolommen wil zien als hij drie jaargangen selecteert, voor elk jaar vier kwartalen.

In feite toont figuur 4.4 twee problemen:

- niet goed leesbare kolomheaders;
- (mogelijk) een inconsistente analyse.

Een goede datumdimensie kan beide problemen oplossen. De datumdimensie moet een kolom KwartaalKey bevatten volgens hetzelfde principe als de smart key die je hebt gemaakt als primary key. Je krijgt dan een kolom met als inhoud bijvoorbeeld 20172, voor het tweede kwartaal van het jaar 2017. Deze sleutel zorgt dat elk kwartaal een unieke sleutel heeft en dus dat elk afzonderlijk kwartaal getoond wordt op rapportages. De sleutel wordt weer als geheel getal opgeslagen in de dimensietabel.

Naast de sleutel kun je ook nog een beschrijvende kolom maken. Hierin kun je bijvoorbeeld de tekst 'Q2, 2017' zetten. Als de gebruiker deze kolom sleept naar een rapport, zullen de headers meteen leesbaar en intuïtief zijn. En dat is goed voor de acceptatie van het datawarehouse.

Een goede datumdimensie bevat voor elke relevante periode een unieke sleutel in de vorm van een getal en een kolom met een leesbare beschrijving.

Bepaal de overige kolommen – Naast de sleutels bevat de datumdimensie beschrijvende informatie. Je moet hierbij denken aan de naam van de dag, het weeknummer waarin die dag valt, de maand van die dag, zowel als nummer als met naam (dus maandnummer is 1 en maandnaam is januari), is de dag een werkdag, enzovoort.

Nu je een idee hebt welke kolommen je moet maken (allemaal uiteraard gebaseerd op de informatieanalyse) kun je de tabel daadwerkelijk maken. Als je op Google zoekt naar 'create date dimension with Excel' vind je veel hits. Ook Kimball heeft een generator voor een datumdimensie, gebouwd in Excel, die je kunt downloaden van zijn site. In de Office 2013 appstore van Microsoft vind je een vergelijkbare generator. Googlen op SQL en date dimension levert ook verschillende hits op voor SQL-scripts die op verschillende platvormen datumdimensies genereren. Op de site van Academic Service vind je, bij dit boek, een T-SQL-script om een datumdimensie te bouwen in SQL Server.

Afhankelijk van waar je een datumdimensie voor nodig hebt, kies je de makkelijkste manier. Voor een relationeel datawarehouse is het uitvoeren van een SQL-script waarschijnlijk het meest voordehandliggend.

▬ Scripts om datumdimensies te genereren zijn van veel plekken te downloaden.

Overige dimensies

Zoals gezegd is de kans zeer groot dat een datamart een datumdimensie zal bevatten. Over de andere dimensies is op voorhand uiteraard veel minder te zeggen. In de gezondheidszorg heb je misschien een patiënt- of cliëntdimensie, bij een commerciële webshop eerder een klantdimensie. Er is echter wel een aantal algemene zaken te noemen met betrekking tot het maken van dimensies. We zullen hier noemen:

- het aantal te maken dimensies;
- denormaliseren van dimensies;
- te kiezen sleutels;
- soort informatie.

Aantal te maken dimensies – Als nadeel van normaliseren is eerder in dit boek genoemd dat er erg veel tabellen ontstaan. Hoeveel dimensies (en dus hoeveel tabellen) heeft een 'goed' stermodel?

Dit antwoord is allereerst volledig afhankelijk van de wensen en eisen. De gevraagde functionaliteit moet geboden kunnen worden. Los daarvan bestaat er een soort vuistregel dat er niet meer dan zeven dimensies per ster moeten bestaan. Het getal zeven komt uit de psychologie. Uit onderzoek blijkt dat mensen zeven zaken tegelijkertijd in hun hoofd kunnen hebben zonder het overzicht te verliezen. Laat iemand zeven plaatjes zien en vraag later terug welke plaatjes het waren, en de persoon is goed in staat de getoonde plaatjes op te noemen. Maak er acht plaatjes van en de kans wordt erg groot dat iemand er één vergeet. Begrip van de datamart is belangrijk voor de acceptatie, vandaar de vuistregel van zeven dimensies.

▬ Probeer het aantal dimensies met stermodel te beperken tot maximaal zeven.

Zeven klinkt erg weinig, zeker als je genormaliseerde databases gewend bent met honderden of zelfs duizenden tabellen. Toch blijkt dat in de praktijk erg mee te vallen. Voor elk proces zijn de eerste vier tot vijf dimensies zo bedacht. Daarna blijkt het al snel lastig te worden nog meer zinvolle dimensies te verzinnen. Vaak zal wat je verzint een attribuut van een al bestaande dimensie zijn. Zeker gezien het feit dat dimensies 'platgeslagen' zijn.

Denormaliseren van dimensies – Misschien is het je opgevallen dat in de datumdimensie zoals eerder besproken, veel redundantie voorkomt. In de dimensie zoals getoond in figuur 4.3 zijn eigenlijk alle kolommen redundant behalve de eerste of de tweede. Als je de DateKey weet, volgen alle andere kolommen daaruit. Voor dimensietabellen

is redundantie normaal. Dimensies zijn platgeslagen, of in andere woorden, gedenormaliseerd.

Dimensietabellen zijn platgeslagen, niet-genormaliseerde tabellen die veel redundantie bevatten.

Voorbeeld

Stel bijvoorbeeld weer dat je de datamart bouwt voor een kleine ICT-opleider. Deze opleider verzorgt cursussen voor zijn klanten. Klanten zijn bedrijven, de cursisten zijn individuen, werknemers bij de klant. In een genormaliseerde database zou je twee tabellen maken, klanten en cursisten. De tabel klanten bevat gegevens over de bedrijven die cursisten sturen en die de facturen betalen. De tabel cursisten bevat NAW-gegevens van de individuele cursisten. In een dimensie voeg je dit samen tot één tabel. Dat wordt een platgeslagen tabel genoemd. Zie figuur 4.5.

Cursist-Key	Naam	Geboorte-datum	Email	Telefoon	Bedrijf	Factuuradres
1	Jan	1-1-1980	Jan@trainsql.nl	06-12345	TrainSQL	Dorpsstraat
2	Piet	2-2-1988	Piet@trainsql.nl	06-54321	TrainSQL	Dorpsstraat
3	Klaas	3-3-1985	Klaas@topbi.nl	06-98765	TopBI	Kerkstraat
...	...	...	...	...	...	...

Figuur 4.5 *Cursistdimensie*

Wat opvalt in deze tabel is dat de informatie met betrekking tot het bedrijf TrainSQL tweemaal is opgeslagen omdat er twee cursisten, Jan en Piet, door TrainSQL op cursus zijn gestuurd. Als er drie cursisten waren gestuurd, waren de gegevens driemaal opgeslagen. Dit is redundant en lijkt daarmee overbodig.

Bedenk echter hoe de tabel gebruikt wordt. Een gebruiker kan de kolom Bedrijf slepen naar een as in een draaitabel of naar een filter in zijn rapport. Automatisch wordt alles van hetzelfde bedrijf samengevoegd. Dus zaken als omzet per klant zijn gemakkelijk (en met drag-and-drop) op te vragen. Er hoeven geen (moeilijke) joins geschreven te worden en de database-engine verspilt geen moeite aan het oplossen van die joins.

Te kiezen sleutels – Het is best practice om dummy keys te introduceren in je datamart. Deze dummy keys worden surrogate keys of warehouse keys genoemd. Het zijn willekeurige nummers, uitgedeeld tijdens het schrijven van nieuwe records in de dimensie, en ze dienen om records uniek te maken.

Een **surrogate key** of **datawarehouse key** is een automatisch gegenereerd, uniek, betekenisloos nummer dat elke record krijgt toegewezen op het moment dat het in het datawarehouse wordt weggeschreven.

Meestal is het van belang om van elk record in de datamart te kunnen herleiden welk record in de operationele database eraan ten grondslag ligt. Daarom bevat een dimensie ook de primaire sleutel van het bronrecord. Dit veld wordt vaak alternate key, source key of, in het Nederlands, bronsleutel, genoemd. In de tabel zoals beschreven in figuur 4.5 zou dit tot twee extra velden leiden: CursistSourceKey en KlantSourceKey.

Soort informatie – Zoals al een aantal keren gezegd, geven de dimensies context aan de feiten. Ze geven de feiten betekenis. Hoe meer beschrijvende velden, hoe groter de meerwaarde van de dimensie. Hoe verhoudt het aantal cursisten met een mbo-achtergrond zich tot het aantal cursisten met een hbo- of wo-achtergrond? Met een attribuut (kolom) vooropleiding in de cursistdimensie, kun je deze vraag beantwoorden. Hoe ver zijn cursisten bereid te reizen voor een cursus en is dat gelijk voor alle cursussen of verschilt dat per cursus? Met een attribuut woonplaats (en een cursusdimensie) is ook deze vraag te beantwoorden.

Hoe meer attributen er zijn, hoe meer analyses je kunt doen. De attributen moeten bij voorkeur tekstuele beschrijvingen zijn en geen codes, tenzij die codes zijn doorgedrongen tot het dagelijks taalgebruik van het bedrijf. Denk maar eens terug aan het verhaal over kwartalen in de datumdimensie. Codes maken slechte kolomheaders op rapporten. En met draaitabellen vormt de inhoud van je kolommen de headers van je draaitabel.

Een dimensie bevat bij voorkeur zoveel mogelijk tekstuele informatie met betrekking tot de dimensie.

Samenvatting

- Dimensies spelen een belangrijke rol bij dimensioneel modelleren. Bij het ontwerpen van de dimensietabellen moet je een paar richtlijnen goed voor ogen houden.
- Gebruik surrogate keys voor alle dimensies, behalve voor de datumdimensie. Gebruik een smart key voor de datumdimensie in de vorm van yyyymmdd. Dus 20170631.
- Neem naast de surrogate key ook de sleutel van het bijbehorende record uit het bronsysteem op.
- Voeg zo veel mogelijk beschrijvende kolommen toe aan de dimensies.
- Ga niet normaliseren maar maak platgeslagen tabellen.
- Beperk het aantal dimensietabellen in een stermodel tot ongeveer zeven.

4.1.4 Slowly Changing Dimensions (SCD)

In hoofdstuk 3 is als een van de mogelijke problemen met rapporten op operationele databases het gebrek aan historische gegevens genoemd. Kimball beschrijft dit als Slowly Changing Dimensions (SCD) en komt met enkele pasklare oplossingen.

De term 'Slowly' refereert hier vooral aan het feit dat sommige gegevens slechts af en toe veranderen. Het gaat om ad-hoc-wijzigingen. Soms kan de frequentie van dit soort wijzigingen groot zijn. Dat heeft impact op de oplossing die je moet kiezen.

Met de term 'Changing Dimension' moet je een beetje oppassen. Eigenlijk wordt bedoeld een wijziging in een dimensieattribuut. Een dimensie bestaat uit vele attribu-

Meestal is het van belang om van elk record in de datamart te kunnen herleiden welk record in de operationele database eraan ten grondslag ligt. Daarom bevat een dimensie ook de primaire sleutel van het bronrecord. Dit veld wordt vaak alternate key, source key of, in het Nederlands, bronsleutel, genoemd. In de tabel zoals beschreven in figuur 4.5 zou dit tot twee extra velden leiden: CursistSourceKey en KlantSourceKey.

Soort informatie – Zoals al een aantal keren gezegd, geven de dimensies context aan de feiten. Ze geven de feiten betekenis. Hoe meer beschrijvende velden, hoe groter de meerwaarde van de dimensie. Hoe verhoudt het aantal cursisten met een mbo-achtergrond zich tot het aantal cursisten met een hbo- of wo-achtergrond? Met een attribuut (kolom) vooropleiding in de cursistdimensie, kun je deze vraag beantwoorden. Hoe ver zijn cursisten bereid te reizen voor een cursus en is dat gelijk voor alle cursussen of verschilt dat per cursus? Met een attribuut woonplaats (en een cursusdimensie) is ook deze vraag te beantwoorden.

Hoe meer attributen er zijn, hoe meer analyses je kunt doen. De attributen moeten bij voorkeur tekstuele beschrijvingen zijn en geen codes, tenzij die codes zijn doorgedrongen tot het dagelijks taalgebruik van het bedrijf. Denk maar eens terug aan het verhaal over kwartalen in de datumdimensie. Codes maken slechte kolomheaders op rapporten. En met draaitabellen vormt de inhoud van je kolommen de headers van je draaitabel.

■ Een dimensie bevat bij voorkeur zoveel mogelijk tekstuele informatie met betrekking tot de dimensie.

Samenvatting

- Dimensies spelen een belangrijke rol bij dimensioneel modelleren. Bij het ontwerpen van de dimensietabellen moet je een paar richtlijnen goed voor ogen houden.
- Gebruik surrogate keys voor alle dimensies, behalve voor de datumdimensie. Gebruik een smart key voor de datumdimensie in de vorm van yyyymmdd. Dus 20170631.
- Neem naast de surrogate key ook de sleutel van het bijbehorende record uit het bronsysteem op.
- Voeg zo veel mogelijk beschrijvende kolommen toe aan de dimensies.
- Ga niet normaliseren maar maak platgeslagen tabellen.
- Beperk het aantal dimensietabellen in een stermodel tot ongeveer zeven.

4.1.4 Slowly Changing Dimensions (SCD)

In hoofdstuk 3 is als een van de mogelijke problemen met rapporten op operationele databases het gebrek aan historische gegevens genoemd. Kimball beschrijft dit als Slowly Changing Dimensions (SCD) en komt met enkele pasklare oplossingen.

De term 'Slowly' refereert hier vooral aan het feit dat sommige gegevens slechts af en toe veranderen. Het gaat om ad-hoc-wijzigingen. Soms kan de frequentie van dit soort wijzigingen groot zijn. Dat heeft impact op de oplossing die je moet kiezen.

Met de term 'Changing Dimension' moet je een beetje oppassen. Eigenlijk wordt bedoeld een wijziging in een dimensieattribuut. Een dimensie bestaat uit vele attribu-

ten. Per attribuut moet je bedenken wat je wilt doen als de waarde (inhoud) van zo'n attribuut verandert. De keuze komt uiteraard voort uit de informatieanalyse omdat de keuze bepaalt welke functionaliteit je krijgt.

De term **Slowly Changing Dimension** refereert aan het feit dat attributen van dimensies in de loop van de tijd kunnen veranderen en draagt standaardoplossingen aan voor hoe hier in het datawarehouse mee om te gaan.

Kimball beschrijft in zijn oorspronkelijke theorie drie manieren om met veranderingen van dimensieattributen om te gaan. Dit zijn:

- SCD type 1
- SCD type 2
- SCD type 3

Als je zoekt op Wikipedia, vind je daar ondertussen acht SCD-types, plus de opmerking dat je de verschillende oplossingen kunt combineren. We zullen hier alleen de drie standaardoplossingen bespreken.

Slowly Changing Dimension type 1

Denk nog eens terug aan de cursistdimensie uit de vorige paragraaf. Een van de kolommen is het e-mailadres van de cursist. Hoewel het een zinvol attribuut is, zal het e-mailadres niet gebruikt worden om in een draaitabel een as te vullen of om gegevens te filteren. Omzet per e-mailadres is geen relevante analyse.

In het bovenstaande voorbeeld is het vooral nuttig om het actuele e-mailadres van cursisten bij de hand te hebben. Als een cursist een nieuw e-mailadres krijgt, overschrijf je het oude adres gewoon met het nieuwe. Dit is hoe veel operationele systemen ook omgaan met wijzigingen. Binnen het dimensioneel modelleren wordt dit een **type 1 Slowly Changing Dimension** genoemd.

Bij **SCD type 1** wordt de oude waarde van een attribuut overschreven door de nieuwe, actuele, waarde. De waarde van het attribuut wordt niet historisch bijgehouden.

Je kunt er natuurlijk ook voor kiezen een wijziging niet door te voeren. Als je het oude e-mailadres behoudt, en negeert dat er een meer actuele waarde voor e-mailadres bestaat, heet dat een type 0 SCD.

Slowly Changing Dimension type 2

Het attribuut Bedrijf in de cursistdimensie levert een ander verhaal op. Als je bijvoorbeeld de omzet per klant (kolom Bedrijf in het voorbeeld) berekent, en je hebt net zoals bij het e-mailadres de oude waarde overschreven door de nieuwe waarde, dan staan alle cursussen die deze persoon heeft gevolgd, vanaf nu geregistreerd bij zijn nieuwe werkgever. Dit is het probleem zoals eerder beschreven in paragraaf 3.7.

Voor dit soort situaties is Slowly Changing Dimension type 2 bedoeld. Bij een SCD type 2 maak je bij elke verandering een compleet nieuw record aan. Nadat Jan van werkgever is veranderd, ziet de cursistdimensie eruit zoals in figuur 4.6. Alle kolommen die iets zeggen over Jan, zoals zijn geboortedatum, veranderen niet. Die worden onveranderd overgenomen. Alleen het bedrijf verandert (en in dit voorbeeld alle kolommen die iets zeggen over de klant).

Cursist-Key	Naam	Geboorte-datum	Email	Telefoon	Bedrijf	Factuuradres
1	Jan	1-1-1980	Jan@trainsql.nl	06-12345	TrainSQL	Dorpsstraat
2	Piet	2-2-1988	Piet@trainsql.nl	06-54321	TrainSQL	Dorpsstraat
3	Klaas	3-3-1985	Klaas@topbi.nl	06-98765	TopBI	Kerkstraat
...	...		...	...	...	...
1001	Jan	1-1-1980	Jan@bedrijfA.nl	06-34567	BedrijfA	Ergens

Figuur 4.6 *Cursistdimensie met SCD 2*

Alle keren dat Jan in het verleden op cursus is geweest, staan vermeld met CursistKey 1. Vanaf nu zal CursistKey 1001 gebruikt worden als Jan op cursus komt. Als je een analyse doet op omzet per klant (Bedrijf) blijft de oude omzet bij het oude bedrijf omdat die omzet hoort bij CursistKey 1. De nieuwe feiten worden gekoppeld aan CursistKey 1001 en vallen dus onder BedrijfA.

Bij **SCD type 2** wordt bij elke verandering van een attribuut een volledig nieuw record aangemaakt. Er bestaan 'actuele' records en 'afgesloten' records.

Merk op dat in de vorige paragraaf is gezegd dat het verstandig is ook de SourceKey (unieke sleutel in het bronsysteem) op te nemen. Door te analyseren op deze SourceKey kun je nog steeds zaken als omzet per cursist opvragen. Zowel omzet per klant als omzet per cursist kunnen nu correct opgevraagd worden, ondanks dat er cursisten zijn die in de loop van de tijd voor andere klanten gaan werken.

Over het algemeen worden bij een SCD type 2 nog wat extra kolommen aan de dimensie toegevoegd. Dat zijn kolommen met een ingangsdatum en een einddatum en wellicht een kolom IsHuidig, die aangeeft of dit record het meest actuele record betreft. Deze kolommen helpen bij het ETL-proces en bij vragen als: hoeveel klanten en hoeveel individuele cursisten hebben wij? Het simpel tellen van records is nu niet meer genoeg!

Slowly Changing Dimension type 3

Stel je categoriseert klanten in de categorieën zilver, goud en platina op basis van de gegenereerde omzet over de afgelopen twaalf maanden. Elke maand kunnen klanten dus in een andere categorie terechtkomen. Op basis van de informatieanalyse is gebleken dat er behoefte is aan inzicht in hoeveel klanten van zilver worden tot goud

of platina en vice versa. Stel voor dit voorbeeld dat je in de eerder gebruikte tabel een kolom Categorie hebt toegevoegd.

Voor de kolom Categorie moet je historie bijhouden anders kun je nooit achterhalen welke, laat staan hoeveel, klanten er per maand van categorie veranderen. Met een SCD type 2 is dit soort vragen wel op te lossen. Alleen levert dat heel complexe, en daarmee trage, query s op. Je moet immers in één en dezelfde tabel voor elk record op zoek naar het meest recente vorige record met dezelfde SourceKey waarvoor geldt dat de kolom Categorie is veranderd. Het meest recente afgesloten record ophalen is al een karwei. Maar dat record kan zijn voor een andere SCD type 2 kolom. Het moet wel gaan om een wijziging van de categorie. Probeer maar eens zo'n query te schrijven.

Voor dit soort vraagstukken is Slowly Changing Dimension type 3 bedacht. Bij type 3 maak je twee kolommen aan voor categorie, de kolom HuidigeCategorie en de kolom VorigeCategorie, alsmede een kolom met daarin de datum van de laatste wijziging. De benodigde informatie zit nu in zijn geheel in het record zelf. Het schrijven van de query is nu voornamelijk het definiëren van de juiste filters. Daarmee is de analyse eenvoudig geworden en tegelijkertijd snel.

Bij **SCD type 3** houd je van een attribuut in twee verschillende kolommen zowel de huidige als de vorige waarde bij. Op die manier kunnen huidige en vorige waarde gemakkelijk vergeleken worden.

Uiteraard zit er wel een beperking aan deze oplossing. Je weet de huidige en de vorige categorie, maar in welke categorie zat de klant daarvoor? Je kunt een derde, een vierde, een vijfde kolom toevoegen, maar dat houdt ergens op. De informatieanalyse moet duidelijk maken of type 3 een goede oplossing is voor de gevraagde informatie of dat deze oplossing te beperkt is.

Samenvatting
Slowly Changing Dimensions bepalen hoe je omgaat met veranderende gegevens. Per dimensieattribuut moet je bepalen of het historisch bijhouden van gegevens nodig is en welke manier het beste aansluit bij de gevraagde functionaliteit.

4.1.5 Conformed dimensions en snowflakes

Als je nog eens terugkijkt naar figuur 4.1 in de inleiding van dit hoofdstuk, zie je naast de tabel dimProduct ook de tabellen dimProductSubcategory en dimProductCategory. Er is hier sprake van een zogenaamde **snowflake** in plaats van een puur stermodel. Kimball spreekt van een snowflake als een of meer dimensietabellen zijn uitgenormaliseerd. In paragraaf 4.1.3 staat dat dimensies zijn platgeslagen en (daardoor) veel redundantie bevatten. Dat is normaal gesproken goed voor de eenvoud en de performance van de datamart.

Een **snowflake** is een stermodel waar van een of meer dimensies niet zijn platgeslagen maar zijn genormaliseerd.

Er kunnen redenen zijn om dimensies uit te normaliseren. De twee redenen die we hier zullen noemen zijn:

- het maken van conformed dimensions;
- beheersbaarheid.

Maken van conformed dimensions

Tot nu toe richt het verhaal zich op het ontwerpen van één ster. Het totale datawarehouse bestaat in de theorie van Kimball uit alle sterren die alle processen beschrijven. Veel dimensies zullen voor meer dan één proces interessant zijn. Je wilt deze dimensies zo veel als mogelijk hergebruiken bij alle relevante processen.

Een eerste stap in het hergebruik van dimensies is het opstellen van een dimensiematrix. In een dimensiematrix zet je de processen uit tegen de dimensies om gemakkelijk te kunnen zien welke dimensies relevant zijn voor welke processen. Figuur 4.7 is een voorbeeld van een dimensiematrix.

Proces	Dimensie			
	Datum	Cursus	Cursist	Locatie
Verkoop	X	X	X	X
Targeting	X	?		X
Evalueren	X	X	X	X

Figuur 4.7 *Een dimensiematrix*

Een **dimensiematrix** laat overzichtelijk zien welke dimensies relevant zijn voor welke processen.

De kruisjes in de matrix laten zien welke dimensie relevant is voor welk proces. Bij Targeting staat echter een vraagteken. Stel dat alle cursussen zijn ingedeeld in curricula, een Business Intelligence-curriculum, een databasebeheer-curriculum, een programmeer-curriculum enzovoort. Stel verder dat bij het bepalen van de doelstellingen voor een jaar niet een doelstelling wordt geformuleerd voor elke individuele cursus, maar een doelstelling per curriculum. Curricula worden echter niet verkocht, het zijn de individuele cursussen die worden verkocht.

Voor beide processen, verkoop en targeting, is de cursusdimensie van belang. De grain van de processen is echter anders. Dit kan een goede reden zijn om informatie omtrent het curriculum niet op te nemen in de dimensie Cursus, maar de curriculuminformatie af te splitsen, te normaliseren. Je krijgt nu een dimensie Cursus en een aparte dimensie Curriculum met een één-op-veel relatie tussen beide.

Voor de ster die behoort bij het verkoopproces heb je nu een paar opties. Je kunt de CurriculumKey opnemen in de cursusdimensie. Je krijgt nu een snowflake. Dit is wat in figuur 4.1 is gebeurd met categorie en subcategorie. Je kunt ook bij elke verkoop zowel een CursusKey als een CurriculumKey opnemen. Er is nu nog steeds sprake van een ster. Combinaties zijn natuurlijk ook mogelijk.

Kimball spreekt in bovenstaand geval over conformed dimensions. Daar wordt niets anders mee bedoeld dan dat een dimensie zo is ontworpen dat hij bruikbaar is

voor elk proces waarvoor hij relevant is. Je moet dus proberen zo generiek mogelijke dimensies te maken.

■ Een **conformed dimension** is een dimensie die dusdanig generiek is opgezet dat hij door elk relevant stermodel, zonder aanpassingen, gebruikt kan worden.

Beheersbaarheid
Denk nog eens terug aan het verhaal over Slowly Changing Dimensions. In de cursistdimensie staat informatie over de individuele cursist, maar ook over het bedrijf (de klant) waarvoor deze persoon werkt. Stel dat tussen deze gegevens SCD type 2-kolommen zitten. Een klant kan misschien wel honderden cursisten gestuurd hebben (denk bijvoorbeeld aan een grote bank waar iedereen op cursus gaat om de nieuwe mogelijkheden van een nieuwe Office-versie te leren kennen). Eén wijziging in een SCD type 2-kolom zou leiden tot honderden nieuwe records. Een aparte dimensie Klant, naast de dimensie Cursist, lijkt nu toch een goed idee.

■ Het normaliseren van een dimensie en daarmee maken van een snowflake kan helpen dimensietabellen klein te houden.

4.1.6 Overige dimensie-overwegingen

Er valt nog veel meer te zeggen over goed dimensieontwerp. Een uitgebreide verhandeling over dimensioneel modelleren vind je in *The Data Warehouse Toolkit* van Kimball en Ross. Hier zullen we onderstaande veelgebruikte constructies en termen nog eens noemen:

- junkdimensie;
- gedegenereerde dimensie;
- unknown member.

Junkdimensies
Stel je weer het opleidingsinstituut van eerdere voorbeelden voor. Dit opleidingsinstituut hanteert een strippenkaart. Vooraf kan een bedrijf tegen leuke korting honderd cursusdagen inkopen. Elke keer dat een cursist een dag op cursus komt, gaat er een strip van het tegoed af. Van elke inschrijving wordt bijgehouden of het betaald is via een strippenkaart of juist niet. Verder heeft elke inschrijving een status: definitief of geannuleerd.

Je kunt in dit geval een dimensie Status maken en een dimensie TypeBetaling. Elke dimensie bevat slechts twee records. Het model wordt complexer dan nodig. De zogenaamde junkdimensie kan uitkomst bieden. Je voegt beide kenmerken samen in een tabel met alle mogelijke combinaties. Vanuit elk feitenrecord verwijs je naar de juiste combinatie. Figuur 4.8 is een voorbeeld van een junkdimensie.

JunkKey	Strippenkaart	Status
1	Ja	Definitief
2	Ja	Geannuleerd
3	Nee	Definitief
4	Nee	Geannuleerd

Figuur 4.8 *Junkdimensie*

Een **junkdimensie** is een dimensie die bestaat uit een combinatie van enkele niet gerelateerde, zeer kleine, dimensies.

Degenerate dimensie

Soms zijn er dimensies die geen attributen hebben. Denk bijvoorbeeld eens aan orders. Het ordernummer is een dimensieattribuut. Als je tot op individuele orders analyses wilt doen en de bijbehorende ordernummers wilt tonen, moeten deze in een dimensie zitten. Dus maak je een dimensie Order. Welke attributen zitten daarin behalve ordernummer?

Alle relevante zaken met betrekking tot de order zijn waarschijnlijk al onderdeel van andere dimensies. Zaken als orderdatum zitten in de dimensie Datum. Alle klantgegevens zitten in de dimensie Klant.

Een degenerate dimensie is een dimensie zonder kolommen zoals de hiervoor genoemde dimensie Order. Vaak wordt hier niet een aparte dimensietabel voor gemaakt. Je kunt het ordernummer opnemen als kolom in de feitentabel.

Een **gedegenereerde dimensie** is een dimensie zonder dimensieattributen.

Unknown en niet van toepassing

In hoofdstuk 3 is als een van de argumenten ter verantwoording van een datawarehouse de soms slechte kwaliteit van gegevens in operationele databases genoemd. Het blijkt in het cursistregistratiesysteem van de opleider uit het voorbeeld voor te komen dat er inschrijvingen bestaan van niet (meer) bekende personen. Als je query's schrijft op een database waarin dat soort zaken voortkomt, moet je je dat goed bewust zijn. Waarschijnlijk moet je een `OUTER JOIN` schrijven om de juiste resultaten te krijgen.

Over het algemeen is dat voor een datamart zeer ongewenst. Drag-and-drop tools genereren waarschijnlijk de verkeerde query's. Om problemen te voorkomen is het normaal om in elke dimensie een extra record op te nemen, de Unknown. Deze krijgt als Key meestal de waarde -1. Het is eigenlijk een dummy record. Hij zorgt ervoor dat ook feiten met ontbrekende dimensiegegevens meegenomen worden in rapporten.

Unknown is een dummy record in een dimensietabel. Dit record zorgt er voor dat feiten uit de feitentabel die niet gelinkt kunnen worden aan een dimensierecord nu toch gelinkt kunnen worden.

Naast dat het juiste dimensierecord onbekend kan zijn, zou het ook kunnen zijn dat een dimensie niet van toepassing is. Stel je hebt een feitentabel met verkopen. Je houdt door middel van een dimensie Verkoper bij wie welke verkopen heeft gerealiseerd. Voor verkopen via het internet is er echter helemaal geen verkoper. Internetverkopen laten verwijzen naar Unknown is raar. Het is helemaal niet onbekend, het is simpelweg niet van toepassing.

In dit soort situaties wordt vaak een tweede dummy record opgenomen in de dimensie. Dit is het record 'Niet van toepassing' en deze krijgt over het algemeen -2 als waarde voor de Key.

4.1.7 Feiten

Bij paragraaf 4.1.3 over dimensies heb ik gezegd dat dimensies misschien wel de belangrijkste component van je dimensioneel model zijn vanwege de context die ze geven aan de feiten. Maar uiteindelijk willen mensen feiten zien. De processen moeten inzichtelijk gemaakt worden. Inzichtelijk houdt in meetbaar en vergelijkbaar. Uit de informatieanalyse zijn indicatoren of zelfs KPI's, kritieke prestatie-indicatoren, naar voren gekomen. Daar wordt een organisatie op gestuurd. Dat zijn de feiten!

Feiten zijn over het algemeen numeriek en meestal aggregeerbaar. Voor een verkoopproces kun je denken aan omzet, marge, kosten, aantallen, korting. Allemaal numerieke grootheden. En de kwartaalomzet is de som, de aggregatie, van drie maandomzetten. Feiten worden ook wel meetwaarden (measures) genoemd.

De feitentabel in het sterschema is de centrale tabel in het midden. In figuur 4.1 aan het begin van dit hoofdstuk is het de tabel factInternetSales. Een feitentabel bestaat uit een verzameling foreign keys en de feiten. De foreign keys (verwijzende/refererende sleutels) verwijzen naar de relevante dimensierecords. In het voorbeeld zijn dat bijvoorbeeld OrderDateKey, ProductKey, CustomerKey, et cetera.

Soorten feiten

Het aggregeerbaar zijn van feiten is een belangrijk aspect. Het hogere management is vaak niet geïnteresseerd in heel gedetailleerde informatie. Zij houden zich meer bezig met de hoofdlijnen en kijken naar bijvoorbeeld omzet op maandniveau. Een teamleider heeft een maandtarget te halen. Om bij te houden of zijn team op schema ligt, houdt hij van dag tot dag de omzet in de gaten. Omzet per dag en omzet tot datum zijn belangrijk. Niet voor alle feiten is het zo gemakkelijk. Er zijn drie soorten feiten:

- additieve feiten;
- niet-additieve feiten;
- semi-additieve feiten.

Additieve feiten – Additieve feiten zijn getallen die optelbaar zijn. Omzet is een mooi voorbeeld van een additief feit. De omzet van alle individueel verkochte producten kan opgeteld worden tot een totaal omzet. Net zo goed kunnen alle maandomzetten opgeteld worden tot een jaaromzet.

Additieve feiten zijn feiten die over alle dimensies zijn op te tellen.

Niet-additieve feiten – Bij de genoemde voorbeelden van feiten staan ook marge en korting. Stel dat je marge opslaat als percentage van de omzet. Vaak wordt binnen organisaties gesproken over de marge in termen van percentages. De marge op dit product is 15%. Als de marge op een ander product 10% is, is dan de totale marge 25%? Werkte het maar zo eenvoudig. Percentages zijn over het algemeen niet op te tellen. Marge is in dit geval een niet-additieve meetwaarde, ofwel een *non-additive measure.*

Naar mijn mening mag je van de gebruikers van de datamart verwachten dat ze de business kennen en weten wat marge is en dat het een percentage is. Ze moeten daarmee snappen dat ze geen percentages moeten optellen. Verwachtingen zijn wel het begin van alle teleurstellingen en de wet van Murphy is altijd van toepassing: dat wat fout kan gaan, zal ook fout gaan. Met een drag-and-droptool die draaitabellen maakt (bijvoorbeeld Excel) worden de cellen (daar waar de feiten in komen te staan) standaard gesommeerd. Excel weet niet dat dat voor een percentage niet de bedoeling is.

Niet-additieve feiten zijn feiten die niet zijn op te tellen, zoals percentages.

In het bovengenoemde voorbeeld is er een eenvoudig alternatief. In plaats van marge als percentage op te slaan, kun je het ook als bedrag opslaan. De marge bedroeg €10,-. Nu is marge wel optelbaar. Een nadeel van deze oplossing is dat in de organisatie marge beschouwd wordt als een percentage. Het datawarehousemodel moet zo goed als mogelijk de werkelijkheid beschrijven, het jargon en de definities van de organisatie gebruiken. De juiste keuze is afhankelijk van de tools die gebruikers krijgen en van het niveau van deze gebruikers.

Semi-additieve feiten – Stel: een bedrijf heeft als doelstelling geformuleerd dat de marge moet verbeteren door de voorraad terug te dringen zonder dat de levertijd langer wordt. De actuele voorraad moet worden bijgehouden. Gisteren waren er tien stuks van product A op voorraad, vandaag zijn dat er acht. Mag je dus concluderen dat er achttien stuks van product A op voorraad zijn? Werkte dat maar zo met je banksaldo!

Stel dat er gisteren van product B ook tien stuks op voorraad waren. Waren er dan in totaal twintig stuks op voorraad? Je kunt je afvragen hoe zinvol het is de voorraad van twee verschillende producten bij elkaar op te tellen, maar het is niet fout. Er zijn twintig stuks goederen op voorraad.

Voorraad is een voorbeeld van een semi-additieve meetwaarde. Over sommige dimensies (bijvoorbeeld de dimensie Product) zijn de getallen wel bij elkaar op te tellen. Over andere dimensies (de dimensie Datum in dit geval) zijn ze niet optelbaar. In de praktijk zal het vaak gaan om de datum/tijddimensie waarover feiten niet aggregeerbaar zijn.

Semi-additieve feiten zijn feiten die over sommige dimensies wel zijn op te tellen maar over andere dimensies niet.

Soorten feitentabellen

Naast verschillende soorten feiten zijn er ook verschillende soorten feitentabellen. De meeste feitentabellen zijn als het ware registraties van wat er is gebeurd. Bij de supermarkt leidt bijvoorbeeld elk piepje dat je bij de kassa hoort tot een regel in de feitentabel. Het feit van de aankoop is geregistreerd en verandert nooit meer. Maar niet alle feiten zijn zo gemakkelijk te vangen. Andere soorten feitentabellen zijn:

- accumulating snapshot;
- periodiek snapshot.

Accumulating snapshot – Bij een supermarkt pak je wat je nodig hebt en je rekent meteen af. Done deal! Als je een auto gaat kopen, vraag je misschien eerst een offerte op. Het is op zich een feit in het hele verkoopproces dat er een offerte is uitgebracht. Er wordt een regel weggeschreven in de feitentabel. Hopelijk leidt de offerte tot een aankoop. Het al bestaande feitenrecord kan nu worden aangevuld met bijvoorbeeld de aankoopdatum. Er is niet zozeer sprake van een nieuw feit als wel van een veranderde status van een al bestaand feit. Omdat de auto nog rijklaar gemaakt moet worden, is er pas enkele dagen later sprake van de echte levering. Wederom moet het reeds bestaande feit worden aangepast.

In het bovenstaande geval, namelijk dat feiten worden aangepast aan een veranderende status binnen het totale proces, wordt er gesproken van een *accumulating snapshot* feitentabel. De feitentabel is een weergave, snapshot, van wat je nu weet. Morgen kan het anders zijn.

Een **accumulating snapshot** is een feitentabel die de huidige status van de feiten weergeeft maar waarbij feiten nog aan verandering onderhevig kunnen zijn.

Periodiek snapshot – Bij de uitleg over semi-additieve meetwaarden hebben we het gehad over voorraad. De registratie van voorraad is een feit. Het is een feit dat er nu acht stuks op voorraad zijn, net zo goed als dat het een feit is dat dat er gisteren nog tien waren. Om inzicht in de voorraad en het voorraadverloop te krijgen, maak je een periodiek snapshot. Elke dag (of elke maand) bekijk je de voorraad en sla je dat op. Figuur 4.9 is een voorbeeld.

DatumKey	ProductKey	BeginVoorraad	AantalBij	AantalAf	EindVoorraad
20130601	1	10	2	5	7
20130601	2	25	10	15	20
20130701	1	7	3	5	9
20130701	2	20	5	5	20

Figuur 4.9 *Periodiek snapshot*

Een **periodiek snapshot** is een feitentabel die de stand van de feiten weergeeft op een specifiek moment. Voor een ander moment wordt een ander snapshot gemaakt.

Het mooie van de opzet van het voorbeeld is dat de kolommen AantalAf en AantalBij aggregeerbaar zijn. Dus het aantal producten dat is binnengekomen in een bepaalde periode is gemakkelijk te berekenen. Tegelijkertijd is van elke periode (maand in het voorbeeld) de stand van de voorraad bekend.

Samenvatting
Een feitentabel is een verzameling sleutels die de feiten relateren aan de bijbehorende dimensierecords. Naast deze sleutels bestaat de feitentabel uit allemaal numerieke kolommen, de feiten.

Het soort proces dat beschreven wordt bepaalt het soort feitentabel. Het soort feit bepaalt hoe het feit gebruikt moet/kan worden in rapportages.

4.1.8 Tot slot

Voorgaande uitleg over dimensioneel modelleren is een uitgebreid verhaal. Tegelijkertijd valt er nog veel meer over te zeggen dan dat je hier hebt gelezen. Als jij degene bent die datamarts gaat ontwikkelen, raad ik je aan *The Data Warehouse Toolkit* van Kimball en Ross te lezen.

Belangrijk binnen de context van dit boek is de rol van het dimensioneel model en vooral die van de datamart. Uit de informatieanalyse volgt wat de organisatie nodig heeft. Wat is het doel van Business Intelligence binnen de organisatie? Welke informatie is er nodig om dat doel te bereiken? Wat voor soort analyses gaat men uitvoeren en met welke KPI's gaat men het proces bijsturen?

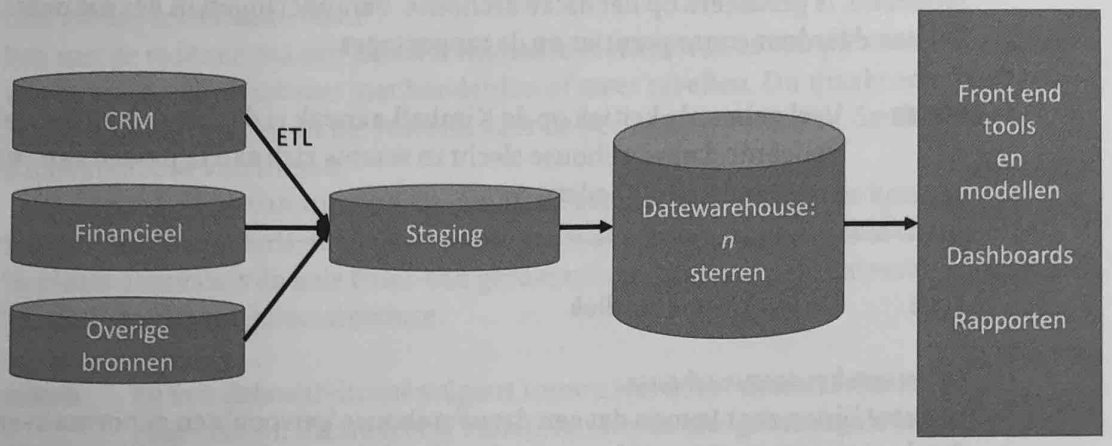
In een gestructureerde Business Intelligence-omgeving, een omgeving die in het BI-volwassenheidsmodel zit bij volwassen of hoger, wordt de informatie ontsloten via datamarts. De opzet van de datamart bepaalt wat de gebruikers straks wel en niet kunnen en hoe intuïtief dat is. Daarmee speelt de datamart een belangrijke rol in de Business Intelligence-architectuur.

4.2 Bill Inmon

Bill Inmon wordt door velen gezien als de vader van het datawarehouse. Hij en Ralph Kimball zijn de twee pioniers op dit gebied. Ze zitten echter niet op één lijn als het aankomt op hoe een datawarehouse eruit moet zien. In deze paragraaf zullen we kort ingaan op de kritiek van Inmon op de strategie van Kimball. Ook zullen we kijken naar wat Inmon zelf voorstelt en de kritiek die daar weer op te geven valt. Welke strategie is beter? Dat is aan ieder zelf ter beoordeling, in dit boek spreken we daar geen waardeoordeel over uit, noch een voorkeur.

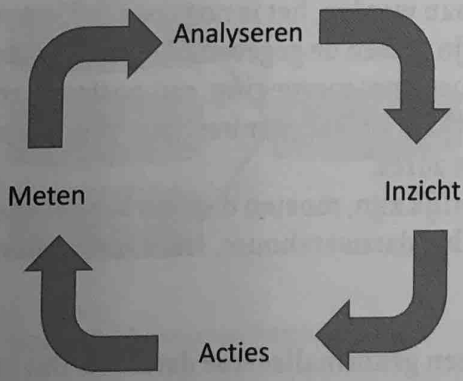
4.2.1 Kritiek op Kimball

Zoals eerder gezegd bestaat in de theorie van Kimball het datawarehouse uit de verzameling van alle sterren. Dimensies worden hergebruikt (conformed dimensies) bij elke feitentabel waarvoor deze relevant is. Voordat de gegevens het datawarehouse ingaan, worden ze in een staging area gezet. Een staging-laag biedt een plek om transformaties (de T uit ETL) toe te passen en geeft flexibiliteit aan het laadproces. Een Kimball-architectuur ziet er in hoofdlijnen uit zoals figuur 4.10.



Figuur 4.10 *Kimball-architectuur*

Bill Inmon is het oneens met de opzet zoals Kimball die voorstaat. Zijn kritiek zit hem voor een groot deel in de Business Intelligence Life Cycle. Figuur 4.11 toont de Life Cycle nogmaals. Business Intelligence heeft tot doel door verbeterd inzicht, via veranderingen, te komen tot verbeteringen. Die veranderingen zouden goed kunnen inhouden dat de processen zelf anders moeten worden. Anders werken, reorganisaties, op andere kenmerken sturen et cetera.



Figuur 4.11 *Business Intelligence Life Cycle*

Wat nu als je net, volgens de Kimball-theorie, een datawarehouse hebt gebouwd en je komt tot het inzicht dat je een of meer van de processen anders moet inrichten? Een ster is gemodelleerd naar het proces, dus de bijbehorende ster moet ook worden aangepast. Misschien dat je andere meetwaarden gaat gebruiken om te sturen. Die meetwaarden hebben op hun beurt misschien andere context nodig om zinvol geana-

lyseerd te kunnen worden. Dimensies moeten dus veranderen. Maar je hebt net veel moeite gedaan conformed dimensies te bouwen zodat dimensies kunnen worden hergebruikt door meer sterren. Wijzigen van een dimensie heeft dus potentieel gevolgen voor alle sterren in het totale datawarehouse.

De kritiek die veel mensen hebben op de opzet van Kimball is dat het vaak lastig is het datawarehouse te laten meegroeien met de veranderende wereld. Kleine aanpassingen in de organisatie kunnen grote invloed hebben op het datawarehouse en daarmee op de hele Business Intelligence. Wat de gebruikers zien, de rapporten en de modellen, is gebaseerd op het datawarehouse. Veranderingen in het datawarehouse hebben daardoor consequenties op de rapportages.

■ Veel gehoorde kritiek op de Kimball-aanpak is dat het dimensioneel gemodelleerde datawarehouse slecht in staat is zich aan te passen aan veranderende omstandigheden.

4.2.2 De Inmon-methodiek

Opzet van het datawarehouse

In grote lijnen zegt Inmon dat een datawarehouse 'gewoon' een genormaliseerde database moet zijn. Met andere woorden, dezelfde ontwerpmethodiek die algemeen aanvaard is voor operationele databases, moet ook worden toegepast voor het datawarehouse.

Het grote verschil tussen normaliseren van databases en het dimensioneel modelleren ervan is het uitgangspunt van wat er gemodelleerd wordt. Bij dimensioneel modelleren wordt voornamelijk het proces gemodelleerd. Denk maar terug aan stap 1 van het dimensioneel modelleren (paragraaf 4.1.2): kies het proces. Het proces staat centraal. Als het proces verandert, verandert het model ook.

Bij normaliseren zijn het de gegevens die gemodelleerd worden. Het gaat er daarbij nog niet om hoe die gegevens gebruikt gaan worden, het is nog geen informatie. Van belang is wat de onderlinge verbanden zijn tussen de gegevens. Een adres bestaat uit een straatnaam, een huisnummer eventueel met toevoeging, een postcode en een plaatsnaam. Het maakt niet uit hoe je werkt of in wat voor business je je bevindt, bovenstaande is altijd waar. Een adres is een adres.

Omdat dit soort verbanden onveranderlijk zijn, moeten deze verbanden als uitgangspunt dienen voor het ontwerp van het datawarehouse. Het datawarehouse wordt daarmee veel stabieler door de tijd heen.

■ Een Inmon-datawarehouse is een genormaliseerde database. Dat betekent dat hij is gemodelleerd op basis van verbanden tussen gegevens die stabiel zijn in de tijd in plaats van op basis van veranderlijke processen.

Een van de tekortkomingen die operationele databases vaak hebben, is het ontbreken van een goede manier om historie bij te houden. De database bevat alleen de actuele gegevens. In paragraaf 4.1.4 heb je uitgebreid kunnen lezen over Slowly Changing

Dimensions, de manier van Kimball om met veranderende dimensieattributen om te gaan. Uiteraard is Inmon zich hier ook van bewust. Maar geschiedenis kan prima genormaliseerd worden.

Omdat in het datawarehouse ook historie wordt opgeslagen, kan de tabelstructuur van het datawarehouse afwijken van die van het bronsysteem. Bovendien worden de gegevens uit meer bronnen gecombineerd, wat tot nog grotere afwijkingen kan leiden.

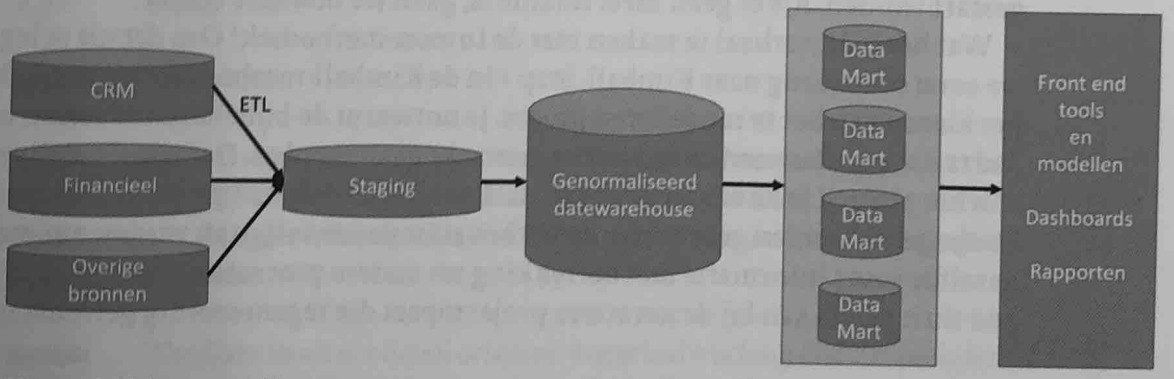
Gebruik van het datawarehouse

Een van de redenen om een datawarehouse te maken, was de complexe tabelstructuur van operationele databases met honderden of meer tabellen. Dit maakt een database lastig om uit te vragen en het is slecht voor de performance. Hoe zit dat dan met het datawarehouse van Inmon?

In de methodiek van Inmon mag niemand aan het datawarehouse komen. Rapporten, dashboards, modellen, ze mogen niet direct uit het datawarehouse lezen. In plaats daarvan is de hele front-end gebaseerd op datamarts. De datamarts worden 'gevoed' vanuit het datawarehouse.

Bij een datawarehouse volgens Inmon wordt het datawarehouse alleen gebruikt om datamarts te vullen, het wordt niet gebruikt als bron voor rapporten of analyses.

Een datamart is een subset van de database gemaakt met een specifiek doel. Meestal is dat doel het ondersteunen van een bepaalde afdeling. Dat vertaalt zich al snel naar een specifiek proces. En om die datamart goed werkbaar, intuïtief en snel te maken, is het een dimensioneel gemodelleerde database.



Figuur 4.12 *Inmon-architectuur*

Gezien dit alles komt een Inmon-architectuur er in grote lijnen uit te zien zoals in figuur 4.12. De hele architectuur, dus alle componenten samen inclusief hier niet getoonde componenten, noemt Inmon de Corporate Information Factory (CIF).

De **Corporate Information Factory** is het geheel van componenten van staging database tot en met datamarts dat ervoor zorgt dat gegevens uit operationele systemen omgevormd worden tot informatie voor de medewerkers van een organisatie.

Als er bij Inmon in de organisatie iets verandert, heeft dat geen impact op het datawarehouse. Het datawarehouse blijft stabiel. In het ergste geval moet de datamart vervangen worden en de rapportages en modellen die daarop gebaseerd zijn ook. Maar dat is logisch verklaarbaar, het proces dat ze beschrijven is immers veranderd, en dus is het acceptabel dat bijbehorende rapporten en databases meeveranderen. Zolang datamarts onafhankelijk zijn van elkaar, blijft de verandering beperkt tot de componenten die met die verandering te maken hebben. Het idee is dat de impact van een kleine verandering klein is, waardoor de totale oplossing beter bestand is tegen de veranderende wereld.

4.2.3 Kritiek op Inmon

Als je de argumenten in deze volgorde leest, denk je misschien dat de Inmon-strategie beter is. Het is niet het doel van dit boek een waardeoordeel te vellen. Doel is de diverse methodieken op hoofdlijnen te vergelijken, zodat je bekend raakt met de verschillen. Je mag zelf een oordeel vellen. Op de Inmon-methodiek is ook de nodige kritiek gekomen. De kritiek zit hem voornamelijk in de opbouw van een Business Intelligence-project en dan voornamelijk op die van het datawarehouse.

Een ideale opzet voor een Business Intelligence-project is een iteratieve opzet. Korte project cycles, zodat snel resultaten getoond kunnen worden aan de organisatie. Vroeger duurde Business Intelligence-projecten maanden en pas helemaal aan het einde kon aan de organisatie getoond worden wat de resultaten waren. Tegenwoordig krijg je geen project meer van de grond als je tegen het management moet zeggen: geef mij een paar ton en een jaar de tijd, over een jaar laat ik zien wat ik heb gemaakt. Wat organisaties willen is met een klein budget en korte doorlooptijd snel meerwaarde zien. Als die meerwaarde er daadwerkelijk is, kan de volgende fase van het project gestart worden. Als er geen meerwaarde is, gaan we ook niet verder.

Wat heeft dit verhaal te maken met de Inmon-methodiek? Om dat uit te leggen gaan we eerst even terug naar Kimball. Stap 1 in de Kimball-methodiek van modelleren was het kiezen van het te modelleren proces. Je ontwerpt de bijbehorende sterstructuur en zodra die af is, kunnen er rapporten gemaakt gaan worden. Dat zijn de 'deliverables' van het project. Je kunt aan de organisatie laten zien wat het project oplevert. Met een beetje geluk worden gebruikers zo enthousiast dat ze zelf gaan vragen om meer, om dezelfde soort informatie met betrekking tot andere processen. Deze manier van werken sluit prima aan bij de iteratieve projectopzet die tegenwoordig gewenst is.

■ De Kimball-methodiek maakt een iteratieve projectaanpak met kleine stappen en snel opleveren van resultaten mogelijk.

Nu terug naar de Inmon-methodiek. Het genormaliseerde datawarehouse omvat eigenlijk alles, gemodelleerd naar de gegevens. Alleen valt het 'alles' niet zo goed te rijmen met de iteratieve projectopzet. Als je slechts de gegevens van bijvoorbeeld het verkoopproces opneemt in het datawarehouse en je wilt later het inkoopproces ook opnemen, kan het zijn dat er structuurveranderingen nodig zijn in het datawarehouse. Maar die hebben impact op de datamarts die al eerder zijn gemaakt.

Bij Kimball is hetzelfde eigenlijk ook waar. De impact is afhankelijk van hoe goed je in staat bent geweest conformed dimensies te maken. Het probleem is bij een Inmon-aanpak over het algemeen groter, wat je meer dwingt richting een 'big bang'-implementatie.

- De Inmon-opzet van een datawarehouse dwingt je eerst het hele datawarehouse te ontwerpen. Dit is moeilijk te rijmen met de tegenwoordig geprevaleerde iteratieve manier van werken.

4.2.4 Samenvatting

Samengevat kun je zeggen dat de Kimball-methode gemakkelijker is in de uitvoering van het project. Het is eenvoudig een kleine scope te kiezen. Daardoor kun je snel resultaten opleveren. Bovendien vergroot je de kans op succes. Het nadeel is dat het moeilijk kan zijn het datawarehouse mee te laten groeien met veranderende eisen en wensen van de eindgebruikers.

De Inmon-methode levert een in de tijd gezien stabieler datawarehouse op. Een bedrijf kan langer profiteren van zijn investering. De grote uitdaging zit in het bepalen van scope en het klein houden van je projecten.

Veel mensen hebben over de jaren heen mengvormen proberen te bedenken. De iteratieve projectaanpak van Kimball en het stabiele datawarehouse van Inmon. De laatste jaren is Data Vault steeds populairder geworden (in Nederland). Data Vault probeert ook het beste van beide aanpakken te combineren.

4.3 Data Vault

4.3.1 Definitie

Zoals gezegd is Data Vault een opzet van een datawarehouse die probeert het beste van de Kimball- en de Inmon-methodiek te combineren. De Data Vault is ontwikkeld door Dan Linstedt. Zelf geeft hij op zijn site (danlinstedt.com/about/data-vault-basics) de volgende definitie:

- *'The Data Vault is a detail oriented, historical tracking and uniquely linked set of normalized tables that support one or more functional areas of the business. It is a hybrid approach encompassing the best of breed between 3rd normal form and star schema.'*

Vrij vertaald staat daar dat een Data Vault bestaat uit genormaliseerde tabellen die gedetailleerde informatie over een of meer deelgebieden van een organisatie bevatten. De tweede zin zegt expliciet dat een Data Vault het beste van zowel normaliseren als van sterschema's bevat.

Data Vault is een verzameling gekoppelde tabellen die gedetailleerde en historische informatie bevat van een of meer processen. Het is een mix tussen normaliseren en dimensioneel modelleren.

De tabelstructuur die Linstedt voorstaat is heel generiek. Een Data Vault is gemakkelijk aan te passen aan nieuwe en veranderde eisen. Het is ook gemakkelijk uit te breiden met nieuwe functionele gebieden. Deze combinatie maakt het mogelijk een Data Vault-structuur op te zetten voor één specifiek proces binnen een organisatie. Later kan deze structuur uitgebreid worden met de gegevens die nodig zijn voor andere processen. Uitgebreid worden is hier iets anders dan aangepast worden. De uitbreidingen hebben geen effect op wat al bestaat.

De Data Vault-structuur levert nog veel meer tabellen op dan het aantal tabellen dat ontstaat bij normaliseren. Gebruikers rechtstreeks laten werken op een Data Vault-structuur is dan ook een slecht idee, zowel qua intuïtiviteit als qua performance. Linstedt stelt dan ook, net als Inmon, voor om het datawarehouse te ontsluiten via een verzameling dimensioneel gemodelleerde datamarts. Inmon heeft zich inmiddels achter de Data Vault geschaard.

4.3.2 Hubs, Links and Satellites

Net zoals bij dimensioneel modelleren is er een stappenplan om tot een Data Vault-structuur te komen. De stappen zijn wel redelijk technisch georiënteerd:

1. Bepaal de business keys, de hubs.
2. Bepaal de relaties tussen business keys, de links.
3. Bepaal de beschrijvende informatie van de business keys, de satellites.
4. Voeg onafhankelijke tabellen toe, zoals een datumtabel en code/beschrijving-tabellen.
5. Voeg, indien nodig, tabellen toe om performance van query's te ondersteunen.

Hubs

De hubs zijn eigenlijk waar het in een organisatie (binnen een proces) over gaat. Hubs zijn het beste te vergelijken met dimensietabellen uit de datamart, maar met twee grote verschillen. Ten eerste bevat een hub-tabel geen beschrijvende informatie maar alleen keys. Ten tweede zijn hubs genormaliseerde tabellen en geen platgeslagen tabellen.

Hubs zijn tabellen die vergelijkbaar zijn met dimensietabellen maar dan zonder alle beschrijvende attributen.

Bij een verkoopproces heb je de hubs order, product, klant, factuur, etc. In het stappenplan wordt gezegd: bepaal de business keys. Gedacht vanuit de operationele database die het verkoopproces ondersteunt, zijn er sleutels (keys) voor deze entiteiten. Waarschijnlijk is er sprake van een uniek productnummer. Net zo goed bestaan er

factuurnummers, klantnummers en ordernummers. Een hub bestaat uit de volgende kolommen:

- Surrogate key, unieke datawarehouse key.
- Business key, de sleutel in de bron.
- Record source, beschrijvend veld dat aangeeft uit welke bron de business key komt.
- Eventueel een Load audit ID, dat verwijst naar een record met metadata over het laadproces, zoals packagenaam en -versie en laaddatum.

Het opvallendste is dat een hub geen andere velden bevat. Dus de hub Product heeft geen velden voor naam, prijs, gewicht, kleur, et cetera. Dat soort velden komen terecht in de satellites.

Links

Een proces wordt gekenmerkt door de interactie die de verschillende entiteiten (de hubs) met elkaar hebben. Deze interactie wordt gevangen in linktabellen. De interacties tussen links worden ook wel transacties of associaties genoemd.

Links zijn tabellen die vergelijkbaar zijn met feitentabellen. Ze beschrijven de relaties (associaties) die verschillende hubs onderling hebben. Ze bevatten de feiten zelf niet.

Een klant ontvangt een factuur. Klant en factuur hebben dus iets met elkaar te maken. Beide zijn hubs, want er is een business key. Of in minder technische terminologie: het zijn primaire entiteiten in het businessproces. De relatie wordt gevangen in een linktabel. Een link bevat de volgende kolommen:

- Surrogate key, unieke datawarehouse key.
- De keys van de hubs, dus hub_klant_key en hub_factuur_key in bovenstaand voorbeeld. Dit zijn foreign keys naar de hubs.
- Record source, beschrijvend veld dat aangeeft uit welke bron de business key komt.
- Eventueel een Load audit ID, dat verwijst naar een record met metadata over het laadproces zoals packagenaam en -versie en laaddatum.

Linktabellen kunnen met feitentabellen worden vergeleken in de zin dat ze de grain bepalen. Meer keys naar hubs opnemen in een link verandert de grain. Voor links geldt net als voor hubs dat er geen beschrijvende velden in zitten. Een link bevat slechts de hierboven benoemde kolommen.

Satellites

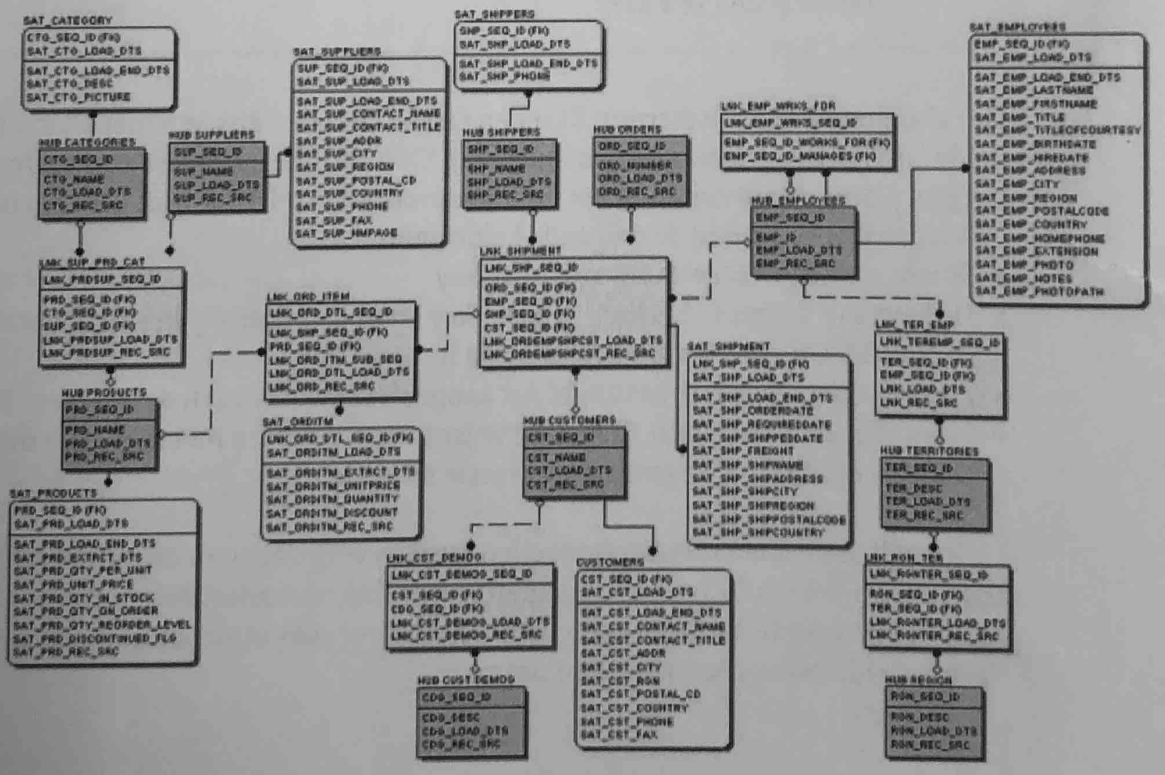
De satellites bevatten uiteindelijk alle beschrijvende kolommen. Dus informatie als kleur en prijs van een product komen in een satellite die een relatie (foreign key) heeft met de hub product. Factuurbedrag is een kolom in een satellite die hoort bij de hub factuur.

Satellites zijn de tabellen die de feiten- en dimensieattributen bevatten. Ze zijn door middel van foreign keys gekoppeld aan links of hubs.

Een hub of een link kan gekoppeld zijn aan meer satellites. Je kunt attributen die uit verschillende bronsystemen komen, opnemen in aparte satellites. Maar ook de frequentie waarmee attributen door de tijd heen kunnen veranderen kan een reden zijn ze samen (of juist niet) in een satellite op te nemen. Een satellite bestaat uit de volgende kolommen:

- Surrogate key, unieke datawarehouse key.
- De key van de hub of de link waar deze satellite bij hoort. Dit is een foreign key.
- Volgnummer, noodzakelijk als er meer geldige versies van de satellite bestaan.
- Begin- en einddatum van deze satellite.
- Record source, beschrijvend veld dat aangeeft uit welke bron de business key komt.
- Eventueel een Load audit ID, dat verwijst naar een record met metadata over het laadproces zoals packagenaam en -versie en laaddatum.
- De 'echte' beschrijvende velden waar het om te doen is.

In figuur 4.13 zie je een voorbeeld van een Data Vault-model zoals het staat op de site van Linstedt. Dit model is gebaseerd op de (oude) voorbeelddatabase van Microsoft, Northwind. Je kunt Northwind nog altijd downloaden van de Microsoftsite. Op de site van Linstedt vind je ook de oorspronkelijke tabelstructuur van Northwind.



Figuur 4.13 *Data Vault-structuur*

4.3.3 Voordelen Data Vault

Dit boek is geen promotieboek voor Data Vault. Voor een uitgebreide lijst met (geclaimde) voordelen verwijs ik naar de site van Dan Linstedt. De volgende punten die je op de site vindt, zullen we hier kort bespreken:

- eenvoud van laden;
- snelheid van laden;
- betrouwbaarheid van laden;
- compliance.

Eenvoud van laden

De tabelopbouw is in een Data Vault heel generiek. Dit maakt dat het gemakkelijk wordt om een Data Vault te vullen. Bij een stermodel of een genormaliseerde database is elke tabel anders. Voor elke tabel moet er goed over nagedacht worden hoe deze te vullen. De query's die nodig zijn zullen elke keer anders zijn.

In de vorige paragraaf is van elk soort tabel beschreven uit welke kolommen die bestaat. Dat is elke keer hetzelfde. Het is relatief gemakkelijk een template te maken voor het laden van een hub, een link, of een satellite omdat ze zo generiek zijn. Een goede template maakt het ontwikkelen van ETL-processen gemakkelijk en snel. Veelal is het automatisch genereren van de ETL zelfs mogelijk. Dat maakt doorlooptijden van projecten korter en verlaagt dus de kosten.

Snelheid van laden

Alle hubs zijn onafhankelijk van elkaar. Dat geldt ook voor links en satellites. De enige afhankelijkheid tussen tabellen zit in de foreign keys die zijn opgenomen in de links en satellites. Dat houdt in dat hubs eerst geladen moeten worden. Verder is er geen afhankelijkheid.

Deze constatering betekent dat er heel veel zaken parallel geladen kunnen worden. Met het huidige aantal cores dat beschikbaar is in servers, kan dat tot heel goede laadprestaties leiden.

Betrouwbaarheid van laden

Data Vault heeft als uitgangspunt dat er geen enkele verandering wordt gedaan op de gegevens. Bedrijfsregels zoals berekeningen, aanvullen van incomplete gegevens en verbeteringen van fouten moet gedaan worden in de datamart. Dat maakt het laden betrouwbaarder en het loggen van fouten eenvoudiger. Dat moet leiden tot een betrouwbaarder laadproces.

Compliance

Door bovenstaande en door de opzet met source keys en verwijzingen naar auditrecords, is het gemakkelijk te voldoen aan verschillende complianceregels.

4.3.4 Samenvatting

Van oorsprong zijn er twee grote datawarehousestromingen: de aanhangers van de Kimball-methodiek en die van de Inmon-methodiek. In de praktijk zijn mensen met beide methodieken tegen beperkingen aangelopen.

Data Vault poogt met een generieke tabelstructuur het beste van de Kimball- en de Inmon-strategie te combineren. Bovendien speelt het in op de krachtiger hardware die we tegenwoordig hebben. Ook houdt het meer rekening met strengere compliance-regels van de laatste jaren.

4.4 Kimball, Inmon en Data Vault vergeleken

In paragraaf 4.2 hebben we de Kimball- en de Inmon-methodologie vergeleken. Die paragraaf eindigde met de verwachting dat Data Vault het beste van beide pakt. Ook nu geldt weer dat we in dit boek geen waardeoordeel vellen over de verschillende besproken strategieën. Het is goed ze allemaal op hoofdlijnen te kennen en begrijpen. De argumenten die we gebruikten om Kimball en Inmon te vergelijken waren:

- stabiliteit van het datawarehouse;
- gemakkelijk iteratief kunnen ontwikkelen.

Stabiel (resilient)

Doordat Data Vault business keys als basis neemt (de hubs) zonder alle bijbehorende rompslomp, ontstaat een stabiel systeem. Als er nieuwe kenmerken ontstaan in de loop van de tijd, of als er nieuwe indicatoren (KPI's) worden bedacht binnen de organisatie, kunnen er simpelweg nieuwe satellites worden bijgemaakt. De bestaande tabellen blijven exact hetzelfde. Query's op de Data Vault hebben dus geen last van de wijzigingen. Maar ook het laadproces heeft er geen last van. Er moet alleen nieuwe code geschreven worden om de nieuwe tabel te vullen.

Veranderingen in de processen hebben geen invloed op de hubs en de bijbehorende satellites. Misschien dat er nieuwe links (met hun eigen satellites) gemaakt moeten worden.

Bovenstaande houdt in dat het Data Vault-model dezelfde (of zelfs een grotere) stabiliteit heeft dan een gewoon genormaliseerd datawarehouse. Dat is wat Inmon wel heeft en Kimball niet.

Iteratief

De Data Vault-structuur is heel generiek. Het is te allen tijde mogelijk het model uit te breiden met nieuwe hubs en links. Dat komt overeen met nieuwe processen toevoegen aan het datawarehouse. Het eenvoudigste Data Vault-model bestaat uit een hub met een satellite. Voor het modelleren van een proces zullen er enkele hubs en links bestaan, elk met hun eigen satellites. Als je het datawarehouse wilt uitbreiden met nieuwe processen, gebruik je de hubs die er al zijn, eventueel aangevuld met nieuwe satellites. Je kunt altijd nieuwe hubs introduceren. Het nieuwe proces komt tot uitdrukking in nieuwe links.

Het voorgaande houdt in dat het gemakkelijk is iteratief te ontwikkelen. Je kunt klein beginnen en dus met een beperkt budget beginnen. Hiermee kun je ook snel resultaten aan de organisatie laten zien. Vandaaruit kun je het datawarehouse en daarmee de totale Business Intelligence-oplossing uitbouwen. Dit is wat Kimball wel heeft en Inmon niet.

4.5 Tot slot

In organisaties die in volwassenheidsmodellen hoog scoren, is het datawarehouse de centrale plek om alle relevante gegevens, ongeacht de herkomst, op te slaan. Vanaf hier gaan de gegevens naar de gebruikers. Bij een Kimball-implementatie wordt het datawarehouse zelf gebruikt. Bij Inmon en Data Vault zit er nog een laag van datamarts tussen het datawarehouse en de gebruikers ervan. Het datawarehouse vormt daarmee het fundament van je Business Intelligence-oplossing.

Dit hoofdstuk was voornamelijk conceptueel van aard en daarmee voor iedereen. Zowel projectleiders als business-analisten als technici hebben baat bij deze achtergrondkennis en -terminologie. In het volgende hoofdstuk leer je meer over de fysieke implementatie van een datawarehouse. Dat hoofdstuk is meer gericht op databaseontwikkelaars en -beheerders.